

MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF REPUBLIC OF MOLDOVA
TECHNICAL UNIVERSITY OF MOLDOVA
FACULTY OF COMPUTERS, INFORMATICS AND MICROELECTRONICS
DEPARTMENT OF SOFTWARE ENGINEERING AND AUTOMATICS

DaQuVa: Data Quality Validator.

DSL

Mentor: prof., Vladislav Crucerescu
Students: Cimendur Rustem, FAF-243
Frunza Damian, FAF-243
Pancenco Mihail, FAF-243
Tureac Daria, FAF-243

Chişinău, 2026

Abstract

Modern relational databases increasingly suffer from data quality problems including inconsistencies, duplicates, missing values, and constraint violations arising from heterogeneous data sources and evolving business rules. Existing solutions, ranging from interactive visual tools and SQL-based rule engines to enterprise data quality platforms and DSL-based pipeline frameworks, either embed validation logic in graphical interfaces or proprietary configurations, or focus on data transformation pipelines rather than on an explicit, reusable, and auditable specification of data quality constraints. This paper presents DaQuVa (Data Quality Validator), a novel domain-specific language designed to address these gaps by providing a declarative, safety-first approach to data quality enforcement in SQL databases. DaQuVa introduces a formally defined grammar, a sequential execution model, and a constrained expression language that makes programs straightforward to review and audit. The language provides core constructs for column-level scanning via pluggable validation tools, threshold-based filtering, local duplicate detection with optional AI-powered similarity scoring, controlled database writes through an explicit safety mechanism (`allowDanger`), result export to multiple sinks, and modular reusability through parameterized functions. A Python-based implementation comprising a regex-driven lexer, a recursive-descent parser, and an execution engine is described. Experimental evaluation demonstrates that DaQuVa can detect and flag erroneous records and near-duplicate entries across realistic datasets with concise, human-readable scripts, while its safety constraints prevent accidental data loss. Unlike general-purpose DSLs or tool-driven approaches, DaQuVa treats data quality rules as a formal, portable contract between technical teams and domain experts, contributing a practical foundation for reproducible and auditable data quality management.

Content

Introduction	5
1 Domain Analysis	6
1.1 Problem statement	6
1.2 Motivation and need for DSL	6
1.3 Target Audience	7
1.4 Existing solutions analysis	8
1.4.1 Non-DSL solutions	8
1.4.2 Existing DSL solutions	12
1.5 Summary of Existing Solutions	15
1.6 Identified Gaps	17
1.7 Functional Requirements	18
1.7.1 Rule Definition and Specification	18
1.7.2 Validation and Execution	18
1.7.3 Modularity and Reusability	18
1.7.4 Integration and Extensibility	18
1.7.5 Usability and Accessibility	19
1.8 Proposed Solution Overview	19
1.9 Use Cases	20
2 DaQuVa DSL: Language Design	22
2.1 Introduction to DaQuVa DSL Design	22
2.2 Initial DSL Grammar	22
2.2.1 Design Principles	22
2.2.2 Lexical Elements	23
2.2.3 Syntactic Grammar in BNF	25
2.2.4 Core Syntactic Structures	32
2.3 Basic DSL Semantics	33
2.3.1 Execution Model	33
2.3.2 Semantic Description of Core Constructs	33
2.3.3 Command Semantics	34
2.3.4 Error Catalogue	38
2.4 Illustrative Example Program	39
2.5 Derivation Trees for Control Structures	40

2.5.1	Variable Definition	40
2.5.2	scan Command	41
2.5.3	filter Command	41
2.5.4	Function Definition	41
2.5.5	merge Command	42
2.5.6	output Command	42
2.5.7	findLocalDuplicates Command	42
2.5.8	newTable Command	42
2.6	Summary	43
3	DSL Implementation	44
3.1	Lexer	44
3.2	Parser	46
3.3	AST	47
3.4	Semantic Analysis and Runtime	51
3.5	Execution Engine	51
3.6	Interactive REPL	52
3.7	Showcase Demo	52
4	Evaluation & Experiments	54
4.1	Experiment 1: Core Data Quality Pipeline	54
4.2	Experiment 2: Latest Language Features	54
4.3	Experiment 3: Robustness and Correctness Checks	54
4.4	Experiment 4: Interactive Use	55
4.5	Experiment 5: Input/Output Proof Summary	55
5	Discussion	57
5.1	Strengths of the Current Prototype	57
5.2	Limitations and Future Work	57
	Conclusions	58
A	Appendix	59
A.1	PBL Problem Map	60
A.2	Project Plan	64
A.3	Team Contract	65
	Bibliography	66

Introduction

Data cleaning is a fundamental process aimed at detecting and correcting errors, inconsistencies, and redundancies in data to ensure high data quality, especially in large-scale systems such as data warehouses, federated databases, and web-based information platforms. Due to the heterogeneity, volume, and continuous evolution of modern data sources, effective data cleaning requires systematic, scalable, and declarative approaches that minimize manual effort while ensuring correctness and reproducibility [1].

Consequently, there is a growing demand for expressive, scalable, and systematic frameworks that enable users to specify, detect, and repair data quality issues in a declarative and reusable manner. This motivates the development of domain-specific abstractions and languages that can bridge the gap between conceptual data quality requirements and their practical implementation in modern data processing pipelines [2, 1].

Interactive data cleaning systems, such as Potter’s Wheel, address the limitations of traditional batch-oriented approaches by tightly integrating discrepancy detection with data transformations. This allows users to iteratively refine cleaning operations on large and heterogeneous datasets through immediate feedback, graphical specifications, and automatic inference of domain structures, reducing the need for complex programming and repeated manual inspection [3].

Wrangler is an interactive system for data transformation that combines direct manipulation of visualized data with automatic inference of applicable transforms. It enables analysts to iteratively clean, reshape, and validate data while capturing an auditable script of all transformations, reducing manual effort and improving reusability [4].

1 Domain Analysis

1.1 Problem statement

Modern data-driven systems rely on large volumes of structured data stored in relational databases, data warehouses, and analytical pipelines. However, real-world datasets are frequently incomplete, inconsistent, duplicated, or semantically incorrect [1, 2].

It is widely recognized that data cleaning and preparation consume more than 50% of the total effort in data science and analytics workflows [1, 2]. This disproportionate cost indicates that data quality management is not a secondary concern but a structural component of modern information systems.

Although relational databases provide basic integrity mechanisms such as primary keys, foreign keys, and type constraints, these mechanisms are insufficient to capture complex domain semantics. Higher-level business rules — including conditional dependencies, cross-table constraints, and semantic validations — are often implemented through ad-hoc SQL queries or embedded procedural scripts.

This fragmented implementation strategy results in:

- Scattered and non-reusable validation logic;
- Tight coupling between validation and execution details;
- Limited auditability and traceability;
- Separation between error detection and correction.

The ISO/IEC 25012 data quality model defines multiple quality characteristics, including accuracy, completeness, consistency, and credibility [5]. However, most practical systems lack a formal and executable specification mechanism aligned with such quality frameworks.

Moreover, as AI-driven systems increasingly rely on structured datasets, the impact of low-quality data becomes amplified. Machine learning models are particularly sensitive to noise, bias, and inconsistencies in input data. Therefore, a systematic and formal approach to data validation is necessary.

1.2 Motivation and need for DSL

The motivation for introducing a Domain-Specific Language (DSL) for data quality validation is grounded in both empirical software engineering research and practical limitations of existing tools.

Empirical studies demonstrate that DSLs improve program structure comprehension and domain alignment compared to general-purpose solutions [6]. By providing domain-native abstractions, DSLs reduce boilerplate code and enhance maintainability.

Current data quality solutions such as interactive cleaning systems [7, 3], configuration-driven validation tools, and research prototypes [8, 9, 10, 11] demonstrate the importance of formalized rule systems. However, they often either:

- Embed rules within configuration files;
- Focus exclusively on detection rather than integrated correction;
- Lack a formally defined and reusable language layer.

Interactive systems such as Wrangler and Potter’s Wheel emphasize mixed-initiative data cleaning but primarily expose rule logic through graphical interfaces rather than portable specifications [7, 3]. Research systems such as NADEEF and HoloClean demonstrate advanced repair strategies but do not focus on DSL formalization as a reusable contract [8, 9].

A DSL tailored for data quality validation provides several advantages:

- Declarative specification of validation rules;
- Clear separation between rule definition and execution;
- Reusability and modularity of quality constraints;
- Compilation into multiple backends (SQL pushdown or streaming scans);
- Integration of detection, suggestion, and correction policies.

Thus, the proposed DSL bridges the gap between low-level SQL validation, configuration-based systems, and interactive cleaning tools by introducing a unified and formally defined language for expressing data quality rules.

1.3 Target Audience

The Data Quality Validator DSL is intended for a multidisciplinary audience involved in data-centric systems.

- **Data Engineers** — who integrate validation rules into ETL and orchestration frameworks. The DSL enables version-controlled and auditable quality specifications;
- **Data Analysts and BI Specialists** — who rely on reliable datasets for reporting and decision support.
- **Data Scientists** — whose workflows are significantly impacted by data preparation costs [2]. The DSL ensures validation before model training;
- **Database Administrators** — who manage relational integrity and require higher-level semantic validation beyond built-in constraints;
- **Domain Experts and Business Stakeholders** — who define business semantics but may not be proficient in SQL. A readable DSL allows them to participate in quality rule definition;
- **Researchers and Academic Community** — who explore formal methods in data validation and DSL design.

By acting as a shared data quality contract, the DSL connects domain semantics with technical execution mechanisms across different stakeholder roles.

1.4 Existing solutions analysis

1.4.1 Non-DSL solutions

Non-DSL solutions represent the historically dominant way of addressing data quality and data cleaning tasks. Instead of offering a dedicated declarative language, these approaches rely on interactive graphical interfaces, visual specification techniques, enterprise platforms, or direct SQL expressions. As a result, they effectively tackle many practical cleaning scenarios, but they tend to encode quality logic in tools, workflows, or queries rather than in an explicit, reusable language of data quality rules.

Visual modeling and UML-based quality approaches.

One direction is to use visual modeling notations such as UML to describe data structures, constraints, and quality requirements on a conceptual level [12]. Such approaches help stakeholders reason about completeness, consistency, and other quality attributes of models, but they do not directly execute validation rules on concrete SQL tables and typically require an additional implementation step in code or database constraints.

Table 1.4.1 - Visual UML-based approaches: pros and cons

Advantages	Limitations
Expressive visual notation for structures and constraints, accessible to architects and analysts. Facilitates discussion of data quality attributes (completeness, consistency) at design time. Supports continuous quality assessment and improvement of models themselves.	Models are descriptive only; quality rules are not directly executable on concrete datasets. Requires a separate implementation step in SQL, application code, or ETL tools, which can diverge from the model. Focuses on model-level quality rather than operational validation of large SQL tables in production.

Interactive visual data transformation tools (Wrangler).

Wrangler is an interactive system for specifying data transformation scripts through direct manipulation of tabular data [7]. Analysts iteratively select parts of the dataset, and Wrangler suggests applicable transformations, previews their effects, and records an auditable transformation history [7]. Although Wrangler internally relies on a declarative transformation language, users mainly see a visual interface for shaping and cleaning data, rather than an explicit, reusable language of quality rules.

Interactive and deterministic data repair (Falcon).

Falcon is an interactive data cleaning system that uses SQL UPDATE queries as the underlying repair language and focuses on deterministic, explainable modifications [10]. It encourages users to iteratively explore the space of candidate repairs, presents them as concrete SQL updates, and evaluates their impact on the data [10]. However, Falcon assumes that users are comfortable reasoning about SQL repairs and

Table 1.4.2 - Wrangler: pros and cons

Advantages	Limitations
Interactive visual interface for specifying complex data transformations without manual scripting.	Focuses on one-off or pipeline-oriented transformations, not on explicit reusable data quality constraints.
Mixed-initiative suggestions help users quickly discover useful cleaning operations.	Underlying transformation language is hidden; rules are not exposed as a stable, human-readable contract.
Maintains an auditable history of transformations that can be replayed.	Primarily designed for pre-analytics data wrangling rather than continuous validation of SQL tables in production.

does not provide a separate, higher-level vocabulary for stating business-level data quality requirements.

Table 1.4.3 - Falcon: pros and cons

Advantages	Limitations
Produces deterministic, explainable repairs in the form of explicit SQL UPDATE statements.	Assumes users can understand and validate SQL-level repair queries, which may be too low-level for domain experts.
Supports interactive exploration of alternative repairs and their impact on the dataset.	Does not define a separate declarative language of data quality rules; repairs are not tied to an explicit rule set.
Integrates naturally with relational databases by operating directly on SQL tables.	Focuses on fixing specific instances of dirty data rather than establishing a reusable, auditable quality contract for future data.

General-purpose interactive cleaning tools (OpenRefine).

OpenRefine is a widely used open-source tool for cleaning and transforming tabular data via a spreadsheet-like interface [13]. It supports faceted browsing, clustering similar values, reconciliation with external data sources, and an undo/redactable history of operations that can be replayed on new datasets [13]. OpenRefine is highly effective for one-off or exploratory data cleaning tasks, but its operations are not designed to serve as a formal and reusable contract of data quality for production SQL databases.

Enterprise data quality platforms.

Large vendors such as IBM, Oracle, or SAS provide comprehensive data quality platforms that combine profiling, cleansing, standardisation, and matching capabilities [1]. These tools typically integrate tightly with existing ETL and MDM solutions, offer rich dashboards, and support complex matching and deduplication rules for customer or product data. They address enterprise-wide governance requirements but come with significant cost and operational complexity.

Table 1.4.4 - OpenRefine: pros and cons

Advantages	Limitations
User-friendly, spreadsheet-like interface suitable for analysts working with messy CSV or JSON data.	Primarily a desktop/GUI tool; not natively integrated as an automated, repeatable check within database pipelines.
Powerful features for clustering similar values, faceted filtering, and reconciliation with external sources.	Cleaning operations are sequences of GUI actions or expressions, not explicit, reusable data quality rules over SQL tables.
Supports undo/redo and operation history that can be applied to similar datasets.	Better suited for ad-hoc or exploratory cleaning than for serving as a formal data quality contract in production environments.

Table 1.4.5 - Enterprise data quality platforms: pros and cons

Advantages	Limitations
Comprehensive feature set, including profiling, cleansing, matching, and enrichment across many data sources.	High licence and infrastructure costs, typically only justified for large organisations.
Tight integration with existing ETL, MDM, and governance tools, enabling end-to-end management of data quality.	Configuration is often complex; setting up rules and workflows requires specialised consultants or trained staff.
Rich dashboards and monitoring for continuous tracking of quality metrics at scale.	Business rules are embedded as configuration artefacts inside the platform, not as a lightweight, portable language over SQL tables.

SQL-based rule engines and database-centric checks.

Another class of solutions relies directly on SQL expressions and stored procedures to implement data quality checks and remediation logic [1]. Teams define validation rules as SQL queries or views that flag invalid rows, and sometimes tie them to scheduling and alerting via ETL or orchestration tools. This approach keeps rules close to the database but can become fragmented and hard to audit as the number of checks grows.

Data profiling and catalog tools with quality rules.

Data catalog and profiling tools provide automated statistics, distributions, and anomaly detection over database tables and columns. Some of them allow users to attach simple quality rules (for example, thresholds on null rates or uniqueness) and to track violations over time in a central dashboard. They are effective for discovering issues and monitoring trends but typically offer only a limited rule language focused on basic metrics.

Self-service analytics tools with built-in validation.

Self-service analytics platforms often include basic validation and cleansing functions within their visual workflows. Business users can define filters, simple constraints, and data type checks directly in a

Table 1.4.6 - SQL-based rule engines: pros and cons

Advantages	Limitations
Uses standard SQL, which is widely known among data engineers and easy to execute in existing databases. Allows fine-grained control and optimisation of checks for specific schemas and performance constraints. Can be integrated into existing ETL pipelines and CI/CD workflows without introducing new runtime components.	Rules are scattered across queries, views, and stored procedures, making them hard to discover and maintain. SQL syntax is verbose for complex business rules and not easily readable by non-technical stakeholders. Rules are not expressed as a single, declarative contract; there is no unified catalogue of quality expectations per table.

Table 1.4.7 - Profiling and catalog tools: pros and cons

Advantages	Limitations
Automatically compute descriptive statistics and distributions that reveal hidden data quality problems. Centralises quality metrics and rule evaluations in dashboards that are accessible to many stakeholders. Help prioritise where to invest cleaning effort by highlighting high-risk tables and columns.	Rule definition is usually restricted to simple thresholds on precomputed metrics (null rate, distinct count, etc.). Do not provide an expressive, table-focused language for complex, cross-column business constraints. Monitoring is often decoupled from actual repair workflows; suggestions for fixes are weak or absent.

drag-and-drop interface and reuse these flows across reports. This reduces the need for ad-hoc scripting but keeps validation logic tied to specific reports rather than to the underlying database tables.

Table 1.4.8 - Self-service analytics tools: pros and cons

Advantages	Limitations
Accessible to non-technical users through visual workflows and predefined cleaning components. Enable rapid creation of reusable cleaning and transformation steps inside analytical pipelines. Reduce dependency on engineering teams for basic validation and filtering tasks.	Validation rules are often bound to particular reports or dashboards, not to shared database schemas. Rule logic remains hidden inside proprietary workflow definitions, hindering auditing and cross-team reuse. Not designed as a general-purpose data quality layer with an explicit, portable specification of rules.

Summary

Overall, non-DSL solutions demonstrate that interactive visual tools, SQL-based repair systems, enterprise platforms, and profiling tools can substantially reduce manual data cleaning effort and improve

transparency of transformations [1, 7, 13]. At the same time, they either lack a dedicated, human-readable language for expressing reusable data quality rules over SQL tables, or they hide such a language behind configuration and GUIs, which makes it difficult to treat these rules as an explicit, auditable contract between technical and business stakeholders [10, 12].

1.4.2 Existing DSL solutions

In contrast to non-DSL approaches, a number of research works and industrial systems propose the use of domain-specific languages (DSLs) for data cleaning, transformation, and data quality management. The core idea behind these approaches is to provide a formal, high-level, and declarative way of specifying data processing logic and quality constraints, closely aligned with domain semantics rather than low-level implementation details.

DSL-based solutions typically aim to improve readability, maintainability, and reusability of data cleaning workflows, while enabling explicit and machine-executable specifications of business rules. Empirical studies indicate that the use of DSLs can significantly increase productivity and reduce error rates in complex data manipulation tasks [6]. As a result, DSLs are increasingly explored as a systematic alternative to ad-hoc scripting and purely tool-driven approaches.

One representative example of such a language is **Jayvee** [11], a domain-specific language and execution framework for modeling data processing pipelines together with integrated data quality logic.

Conceptually, Jayvee treats data processing as a formally defined *pipeline model*. A pipeline is a strictly ordered sequence of computational units called *blocks*, where the output of each block becomes the input of the next one. This enforces an explicit, linear dataflow model that makes transformations, dependencies, and execution order unambiguous.

Jayvee defines three fundamental categories of blocks:

- **Extractor blocks** — model data sources (e.g., HTTP endpoints, filesystems, databases) and introduce data into the pipeline;
- **Transformer blocks** — perform structured transformations, filtering, selection, and interpretation of intermediate data representations;
- **Loader blocks** — represent data sinks and persist processed data into storage systems.

This architecture maps directly to the classical ETL (Extract–Transform–Load) paradigm, but formalizes it as a typed, declarative pipeline language rather than as scripts or tool configurations.

A central design element of Jayvee is its *type system based on value types*. Instead of treating data as untyped strings or generic table cells, Jayvee allows users to define domain-specific value types (e.g., percentages, identifiers, measurements) and attach *formal constraints* to them. These constraints define admissible value ranges, formats, and semantic properties, and are evaluated automatically during pipeline execution. As a result, data quality validation becomes an intrinsic part of the execution semantics, not an

external checking step.

Jayvee also supports *explicit transformation definitions* that map values between types, enabling controlled and traceable semantic conversions (e.g., unit conversions, format normalization). Furthermore, it allows the definition of *composite block types*, which are higher-level abstractions composed of multiple basic blocks. This enables modularization, reuse, and abstraction of complex processing logic into reusable components.

From a software engineering perspective, Jayvee can be classified as a *typed dataflow DSL* that exhibits:

- an explicit and structured pipeline model;
- formally specified computational units;
- a domain-oriented type system;
- built-in constraint-based data validation;
- mechanisms for modular composition and reuse.

Consequently, Jayvee transcends traditional transformation scripting and serves as a formal modeling language for data pipelines with integrated data quality semantics.

Table 1.4.9 - Jayvee: advantages and limitations

Advantages	Limitations
Formal pipeline model with explicit dataflow semantics and execution order.	Requires learning a new language and modeling paradigm compared to SQL-based solutions.
Strong domain-oriented type system with value types and constraints.	Less flexible for highly dynamic or unstructured data compared to script-based approaches.
Integrated data quality validation as part of execution semantics.	Primarily focused on structured pipelines; not designed for ad-hoc exploratory cleaning.
Clear separation of extraction, transformation, and loading stages.	Depends on predefined block types for integration with external systems.
Support for modular composition via composite blocks and reusable components.	Smaller ecosystem compared to mature enterprise ETL platforms.
Declarative, readable specification of complex data workflows.	Requires formal modeling effort even for simple pipelines.

Grafter and Grafterizer.

Another representative DSL-based approach is **Grafter**, together with its interactive environment **Grafterizer** [14]. Grafter is implemented as an embedded domain-specific language (eDSL) in the functional programming language Clojure, targeting data cleaning, transformation, and semantic data publishing workflows. While syntactically realized as a software library, Grafter introduces a dedicated abstraction layer for expressing domain-level data transformation pipelines, thereby fulfilling the defining characteris-

tics of a DSL.

Conceptually, Grafter follows a *functional pipeline model*, where data transformations are represented as composable functions, called *pipes*. Each pipe performs a single, well-defined transformation step, and complex workflows are constructed through functional composition. This model enforces a clear and explicit dataflow structure, in which intermediate results are immutable and transformation semantics are deterministic, enabling transparent reasoning about pipeline behavior.

A distinctive feature of Grafter is its strong integration of *data cleaning and semantic transformation*. In addition to conventional tabular cleaning operations such as filtering, normalization, enrichment, and value derivation, Grafter provides first-class constructs for mapping cleaned data into RDF representations. Users define semantic mappings using explicit triple generation rules, enabling the transformation of raw tabular data into Linked Data formats suitable for publication and semantic interoperability.

Grafterizer extends the Grafter DSL by providing an interactive graphical environment for constructing, testing, and executing transformation pipelines. Through a visual interface, users can compose pipelines from predefined transformation operators, preview intermediate results, and iteratively refine workflows. This interactive layer lowers the barrier of entry for domain experts while preserving the expressive power and compositional semantics of the underlying DSL.

From a software engineering perspective, Grafter can be classified as a *functional embedded DSL* characterized by:

- compositional transformation pipelines;
- functional abstraction and immutability;
- explicit dataflow modeling;
- extensible operator libraries;
- integrated semantic data mapping.

This design makes Grafter particularly suitable for large-scale data transformation and semantic publishing workflows, while Grafterizer facilitates interactive development and validation of data cleaning logic.

Wrangle (Trifacta / Alteryx).

Wrangle is a proprietary domain-specific language used internally by the Trifacta (now Alteryx Designer Cloud) platform to specify data transformation recipes [15]. Although users primarily interact with Wrangle through a graphical interface, all transformation logic is ultimately compiled into a sequence of formal Wrangle expressions. As such, Wrangle represents a *hidden DSL*, whose syntax and semantics are abstracted away from direct user interaction.

Conceptually, Wrangle follows a *stepwise transformation model*, where a dataset is transformed through an ordered sequence of transformation steps called *recipes*. Each recipe step corresponds to a formal transformation operator (transform) applied to selected columns and rows, optionally parameterized

Table 1.4.10 - Grafter and Grafterizer: advantages and limitations

Advantages	Limitations
Functional pipeline DSL enables modular, compositional data transformations.	Requires understanding of functional programming concepts, which may be unfamiliar to many data analysts.
Integrated support for both data cleaning and RDF generation within a single language framework.	Strong orientation toward semantic web workflows limits applicability to purely relational or analytical pipelines.
Clear and explicit dataflow semantics improve reasoning about transformation correctness.	Embedded DSL syntax inherits complexity from the host language (Clojure).
Interactive graphical interface supports incremental pipeline development and debugging.	Graphical abstraction may obscure underlying transformation semantics for advanced users.
Extensible through custom transformation operators and semantic mappings.	Custom extensions require proficiency in Clojure and JVM tooling.
Supports semantic interoperability and Linked Data publishing.	More complex deployment and runtime environment than pure SQL-based validation solutions.

by expressions, functions, and predicates. This structure defines a linear dataflow pipeline, in which each step incrementally modifies the dataset state.

The Wrangle language provides a rich collection of built-in operators and functions covering string processing, numerical computation, date handling, statistical aggregation, window operations, and nested data manipulation. Transformation expressions combine column references, functional operators, and logical predicates, enabling concise yet expressive specification of complex cleaning operations. However, direct authoring of Wrangle scripts is intentionally restricted; instead, users construct recipes via interactive selection and manipulation, with the system generating the corresponding DSL code implicitly.

From a software engineering perspective, Wrangle can be classified as a *proprietary embedded DSL for data transformation pipelines* characterized by:

- linear recipe-based pipeline structure;
- operator-centric transformation semantics;
- expression-based column and row selection;
- integrated functional computation model;
- tight coupling to a graphical interaction environment.

While this design significantly lowers the entry barrier for non-technical users, it also limits transparency, portability, and independent reuse of transformation logic outside the hosting platform.

1.5 Summary of Existing Solutions

Existing solutions for data cleaning and data quality management can be broadly classified into **non-DSL approaches** and **DSL-based approaches**.

Non-DSL approaches, including visual modeling (e.g., UML-based quality assessment), interactive

Table 1.4.11 - Wrangle DSL: advantages and limitations

Advantages	Limitations
Rich set of built-in transformation operators and functions for diverse cleaning tasks.	Proprietary and closed language, not independently reusable outside the Alteryx ecosystem.
Interactive construction of pipelines lowers the entry barrier for non-technical users.	Transformation logic remains hidden, reducing transparency and auditability.
Formal underlying DSL enables deterministic and reproducible execution of workflows.	Users cannot directly author or maintain Wrangle code as a first-class artifact.
Supports complex nested, statistical, and window-based transformations.	Strong vendor lock-in; workflows are not portable across systems.
Seamless integration with enterprise analytics and cloud execution environments.	Primarily designed for interactive analytics, not for explicit specification of reusable data quality contracts.

data transformation tools (e.g., Wrangler, Falcon, OpenRefine), enterprise platforms (e.g., IBM, Oracle, SAS), and SQL-based rule engines, primarily rely on either graphical interfaces, ad-hoc scripts, or embedded configurations to perform data cleaning and validation [1, 7, 13, 10, 12]. These solutions offer several practical advantages:

- Facilitate iterative and interactive cleaning processes;
- Reduce manual effort for common data transformations;
- Integrate with existing ETL pipelines and enterprise workflows;
- Provide audit trails or histories of applied transformations.

However, they also exhibit significant limitations:

- Data quality rules are often hidden, ad-hoc, or fragmented across queries, workflows, or GUI actions;
- Lack of a formal, human-readable, and reusable specification of business-level quality rules;
- Limited support for automated validation and modular composition of rules.

DSL-based approaches, such as Jayvee [11] and Grafter/Grafterizer [14], introduce domain-specific abstractions for modeling pipelines and composable transformations. They formalize aspects of data processing through explicit constructs, typed value systems, and built-in validation mechanisms. The advantages of DSL-based solutions include:

- Explicit and structured representation of transformations and dataflow;
- Integrated constraint-based validation and semantic reasoning;
- Support for modular composition and reuse of processing units.

Their limitations include:

- Steeper learning curve due to new languages and paradigms;
- Focus on transformation pipelines rather than purely on data quality contracts;
- Smaller ecosystems compared to mature enterprise ETL platforms.

Overall, while existing solutions reduce manual data cleaning effort and increase transparency of transformations, they generally do not provide a dedicated, formal language for specifying *auditable, reusable, and declarative data quality rules* over relational tables. This gap motivates the development of a dedicated DSL for data quality validation.

1.6 Identified Gaps

Despite the diversity of existing solutions for data cleaning and data quality management, several limitations remain that motivate the development of a dedicated domain-specific language (DSL):

- **Lack of explicit, reusable rules:** Non-DSL tools often embed validation logic in GUI actions, scripts, or platform configurations, making it difficult to extract, audit, and reuse business-level data quality rules;
- **Fragmented validation:** SQL-based and enterprise approaches frequently scatter checks across multiple queries, stored procedures, or pipelines, complicating maintenance, discoverability, and traceability;
- **Limited formal semantics:** Most visual or script-driven solutions do not provide formal semantics for transformations or data constraints, which hinders automated reasoning, error detection, and reproducibility;
- **Low modularity and composability:** Existing solutions often lack a mechanism for defining modular, composable transformations or quality rules that can be reused across datasets and projects;
- **High dependence on technical expertise:** Tools such as Falcon, Wrangler, and Grafter assume familiarity with SQL, functional programming, or pipeline modeling, which can exclude domain experts from participating directly in quality specification;
- **Insufficient integration of semantic validation:** Only a few DSL-based approaches (e.g., Grafter) combine data cleaning with semantic or type-based validation, leaving a gap in end-to-end formalized data quality enforcement;
- **Steep learning curves and ecosystem limitations:** DSL-based solutions often require adoption of new languages and paradigms, and their smaller ecosystems make integration, tooling, and community support limited compared to mature enterprise platforms.

These gaps indicate that while existing approaches improve efficiency and provide partial solutions, there is no unified, formal, and accessible mechanism to specify, enforce, and audit data quality rules in a reusable and domain-oriented manner. Addressing these gaps motivates the design of a dedicated *Data Quality Validator DSL*, which aims to combine the formal rigor of DSL-based solutions with the usability and accessibility required for widespread adoption by both technical and domain experts.

1.7 Functional Requirements

The Data Quality Validator DSL aims to provide a formal, expressive, and user-friendly language for specifying, validating, and enforcing data quality rules. The functional requirements of the DSL can be grouped into the following categories:

1.7.1 Rule Definition and Specification

- **Declarative rule syntax:** Users should be able to express data quality rules in a clear, human-readable, and declarative form;
- **Support for multiple data types:** The language must allow rules over standard types such as integers, decimals, strings, dates, and domain-specific types (e.g., percentages, IDs);
- **Constraint specification:** Users can define constraints on values, ranges, formats, uniqueness, relationships between columns, and custom predicates;
- **Composite rules:** The DSL should support composing multiple simple rules into complex, reusable rulesets.

1.7.2 Validation and Execution

- **Table-level validation:** The DSL must allow rules to be applied over entire tables or datasets;
- **Row-level and column-level checks:** Users can define rules that operate at granular levels (per row, per column, or across multiple columns);
- **Deterministic evaluation:** Rule execution should produce deterministic results, ensuring reproducibility;
- **Error reporting and diagnostics:** The system must provide clear feedback on violations, including location (row/column) and the rule that failed;
- **Batch and incremental validation:** The DSL should support validating historical datasets as well as streaming or newly appended data.

1.7.3 Modularity and Reusability

- **Reusable rule libraries:** Users should be able to define, store, and reuse commonly applied rules across datasets and projects;
- **Parameterized rules:** The language should allow parameterization to apply the same logic to different columns, tables, or datasets without rewriting rules;
- **Rule grouping and categorization:** Rules can be grouped by domain, dataset, or quality attribute for easier management and auditing.

1.7.4 Integration and Extensibility

- **Database connectivity:** The DSL must support reading from and writing to relational databases (e.g., SQL-based systems) and potentially flat files (CSV, JSON);

- **Extensible functions and predicates:** Users can define custom functions or predicates to handle specialized validation logic;
- **Interoperability:** Rulesets can be exported, imported, or integrated with existing ETL pipelines and monitoring systems.

1.7.5 Usability and Accessibility

- **Readable syntax:** Rule expressions should be intuitive to domain experts with minimal programming background;
- **Interactive development support:** Optional support for IDE-like features, auto-completion, and in-line validation to facilitate rule creation;
- **Auditability:** All rule definitions, executions, and violations should be logged for auditing and compliance purposes.

In summary, the Data Quality Validator DSL must combine *formal rigor*, *reusable abstractions*, and *user accessibility*, addressing the limitations of both non-DSL and existing DSL solutions, while enabling automated, deterministic, and auditable enforcement of data quality rules.

1.8 Proposed Solution Overview

The proposed Data Quality Validator DSL is a declarative language designed to define, execute, and manage data quality rules efficiently. It allows users to write high-level scripts that specify constraints, validations, and transformations on datasets in a human-readable and reusable format.

Example: Column-level validation

The following DSL snippet ensures that the ‘age’ column contains only positive integers:

```
validate column "age" {
  type: integer
  condition: value > 0
  on_error: "Age must be positive"
}
```

Example: Cross-column rule

The next snippet ensures that ‘end_date’ is always later than ‘start_date’ :

```
validate row {
  condition: end_date > start_date
  on_error: "End date must be after start date"
}
```

Example: Referential integrity check

This example enforces that ‘customer_id’ exists in the ‘customers’ table :

```

validate column "customer_id" {
  reference: "customers.id"
  on_error: "Customer ID not found"
}

```

Example: Reusable rule definition

Rules can be parameterized and reused across datasets:

```

rule positive_number(column_name) {
  validate column column_name {
    type: integer
    condition: value > 0
    on_error: "Value must be positive"
  }
}

```

```

apply positive_number("age")
apply positive_number("salary")

```

These examples demonstrate the DSL's **readability, reusability, and expressiveness**, allowing both domain experts and engineers to define data quality rules without low-level SQL or procedural code.

1.9 Use Cases

The DSL supports a variety of practical scenarios:

1. Batch validation of datasets

```

dataset "employees.csv" {
  validate column "email" {
    pattern: /^[a-z0-9._%+-]+@[a-z0-9.-]+\.[a-z]{2,}$/
    on_error: "Invalid email format"
  }
  validate column "salary" {
    condition: value >= 0
    on_error: "Salary must be non-negative"
  }
}

```

2. Incremental validation for streaming data

```

stream "transactions" {

```

```

validate row {
  condition: amount > 0
  on_error: "Transaction amount must be positive"
}
}

```

3. Aggregated checks and group-level rules

```

validate group "orders" by "customer_id" {
  condition: sum(amount) <= credit_limit
  on_error: "Customer exceeded credit limit"
}

```

4. Logging and auditing

```

log "validation_report.log" {
  include: all_errors
  format: csv
}

```

These use cases illustrate how the DSL can be applied to column-level, cross-column, group-level, and streaming validations, while supporting modular, auditable, and reusable rule definitions. Users can write scripts like these to enforce consistent and high-quality datasets across multiple projects.

2 DaQuVa DSL: Language Design

2.1 Introduction to DaQuVa DSL Design

DaQuVa (Data Quality Validator) is a domain-specific language designed to facilitate the safe and efficient cleaning, validation, and analysis of large datasets within SQL databases. The language adopts a declarative programming paradigm, emphasizing "what" to do rather than "how" to do it, which aligns with the needs of data engineers and analysts who require concise, readable scripts for data quality tasks.

The DSL is tailored for post-validation data cleaning in enormous databases, where data has already undergone initial validation but may still contain inconsistencies, duplicates, or errors. DaQuVa ensures cautious operations by incorporating confidence-based scoring (e.g., `dirty_level`, `duplicate_score`) and explicit safety mechanisms (e.g., `Here()` with `allowDanger`) to prevent accidental data loss.

Key design principles include:

- **Declarative Nature:** Users describe data transformations and validations without specifying low-level implementation details;
- **Safety-First:** Destructive operations require explicit thresholds and permissions;
- **Reusability:** Support for variables and functions to enable modular, pipeline-friendly scripts;
- **Extensibility:** Architecture allows integration of AI-based or rule-based tools for validation.

The implemented prototype was extended beyond the initial grammar with several user-facing features that are important for practical data exploration. The filter language now supports compound boolean expressions with explicit AND / OR precedence, string predicates `starts_with`, `contains`, and `ends_with`, result-set ordering through `sort`, row capping through `limit`, and an interactive REPL for incremental execution. Duplicate detection also accepts `levenshtein_matcher` as an alias for the fuzzy duplicate matcher, making the matching strategy clearer in scripts that are specifically based on Levenshtein distance.

The prototype additionally introduces a `summarize` command that produces a compact, aggregate *proof* table over one or more pipeline stages. Rather than appending more annotation columns to the data, it collapses each named stage into a single row of metrics — row count, column count, duplicate-group and duplicate-candidate counts, typo-suspect count, and the number of input rows represented after a merge — so that the effect of a cleaning pipeline (for example, "30 raw rows reduced to 15 merged rows") can be exported and verified at a glance.

2.2 Initial DSL Grammar

2.2.1 Design Principles

The grammar of DaQuVa is guided by three principles:

- **Readability.** Statements read like short English instructions (`scan table."col" using tool "name";`);

- **Minimalism.** Only constructs required for data-quality tasks are included; general-purpose programming facilities are deliberately omitted;
- **Safety by default.** Destructive operations on the underlying database require an explicit `allowDanger` flag, making accidental data modification syntactically impossible.

2.2.2 Lexical Elements

The lexical layer defines the primitive tokens consumed by the parser.

Keywords

The reserved words of DaQuVa are grouped by functional category in Table 2.2.1. They may not be used as identifiers.

Table 2.2.1 - Reserved keywords of DaQuVa, grouped by category.

Category	Keywords
Database	<code>my_db</code>
Function & Control	<code>function</code> , <code>return</code>
Commands	<code>scan</code> , <code>filter</code> , <code>merge</code> , <code>output</code> , <code>newTable</code> , <code>findLocalDuplicates</code> , <code>sort</code> , <code>limit</code> , <code>summarize</code>
Conditions & Logic	<code>where</code> , <code>AND</code> , <code>OR</code> , <code>columns</code> , <code>starts_with</code> , <code>contains</code> , <code>ends_with</code>
Ordering	<code>by</code> , <code>asc</code> , <code>desc</code>
Tool Integration	<code>using</code> , <code>tool</code>
Output Targets	<code>console</code> , <code>file</code> , <code>from</code> , <code>addColumnns</code>
Safety Flags	<code>Here</code> , <code>allowDanger</code>

Identifiers

An identifier begins with a letter and may be followed by any combination of letters, digits, and underscores:

```

<identifier> ::= <letter> { <letter> | <digit> | "_" }
<letter>     ::= "a" | ... | "z" | "A" | ... | "Z"
<digit>      ::= "0" | ... | "9"

```

Literals

DaQuVa supports two literal types:

```

<string> ::= ''' { <any_char_except_quote> } '''
<number> ::= <digit> { <digit> }

```

<number> denotes a non-negative integer, used for thresholds and confidence scores. String literals are delimited by double quotes and may contain any character except an unescaped double quote.

Comments

Single-line comments begin with # and extend to the end of the line. Comments are ignored by the parser.

Whitespace and Layout

Unlike indentation-sensitive languages such as Python, DaQuVa treats whitespace purely as a lexical separator. The parser ignores formatting characters and does not assign semantic meaning to indentation.

The following rules apply:

- One or more whitespace characters may appear between tokens to separate them;
- Consecutive spaces are treated as a single separator;
- Tab characters and newline characters are ignored by the parser;
- Program structure is defined by explicit punctuation rather than indentation;
- The semicolon (;) is the only required statement terminator and separates commands.

As a result, developers may freely format DaQuVa programs for readability. The following examples are syntactically equivalent:

```
filter potential_dirty where dirty_level > 70;
```

```
filter    potential_dirty
  where   dirty_level    >    70    ;
```

This design follows the convention of languages such as C and SQL, where whitespace improves readability but does not influence program semantics.

Operators and Punctuation

Table 2.3.1 lists the core semantic entities of DaQuVa.

Table 2.2.2 - Core semantic entities of DaQuVa.

Entity	Description
my_db	Declares the active database connection using an opaque connection string. Exactly one my_db declaration is expected per program; re-declaration overwrites the current context.
Table	The primary data container. Tables exist either in the connected database or as in-memory result sets produced by scan or newTable.
Variable	Any command result can be bound to a variable. Variables are untyped at the syntax level but carry a <i>result-set</i> value at runtime.
Tool	An external validator or AI-powered analyser identified by a string name (e.g., "fuzzy_matcher", "typo_checker"). The runtime resolves the name to a registered tool implementation.
Column / Meta-variable	Columns such as dirty_level and duplicate_score are injected into result sets automatically by scan and findLocalDuplicates, respectively. They may also be added explicitly via addColumns.
Function	A named, reusable sequence of commands parameterised by identifiers. Calling a function executes its body in a new scope; parameters shadow outer variables.
Threshold / Confidence	An integer in the range [0,100] representing a minimum quality score. Used in filter conditions and as the confidence argument to Here().
Here()	A special sink that redirects writes into the live database. Requires an optional confidence argument; when used with merge, it writes merged records back to the source.
allowDanger	A boolean flag that must be present on any merge command that targets Here(). It serves as an explicit acknowledgement that the operation modifies persisted data.

2.2.3 Syntactic Grammar in BNF

The complete context-free grammar of DaQuVa is given below. Non-terminals are enclosed in angle brackets; terminals are quoted. Curly braces denote zero-or-more repetition; square brackets denote an optional element.

Listing 2.2.1 - Complete BNF grammar of DaQuVa.

```
<SingleSpace> ::= " "
```

$\langle \text{OneOrMoreSpaces} \rangle ::= \langle \text{SingleSpace} \rangle$
 $\quad \quad \quad | \langle \text{OneOrMoreSpaces} \rangle \langle \text{SingleSpace} \rangle$

$\langle \text{opt-spaces} \rangle ::= \langle \text{OneOrMoreSpaces} \rangle$
 $\quad \quad \quad | \langle \text{empty} \rangle$

$\langle \text{program} \rangle ::= \langle \text{opt-spaces} \rangle \langle \text{statement} \rangle \langle \text{opt-spaces} \rangle \langle \text{program} \rangle$
 $\quad \quad \quad | \langle \text{empty} \rangle$

$\langle \text{empty} \rangle ::=$

$\langle \text{statement} \rangle ::= \langle \text{opt-spaces} \rangle \langle \text{db_init} \rangle \langle \text{opt-spaces} \rangle$
 $\quad \quad \quad | \langle \text{opt-spaces} \rangle \langle \text{variable_assign} \rangle \langle \text{opt-spaces} \rangle$
 $\quad \quad \quad | \langle \text{opt-spaces} \rangle \langle \text{function_def} \rangle \langle \text{opt-spaces} \rangle$
 $\quad \quad \quad | \langle \text{opt-spaces} \rangle \langle \text{command_statement} \rangle \langle \text{opt-spaces} \rangle$

$\langle \text{db_init} \rangle ::= \text{"my_db"} \langle \text{string} \rangle \text{";"}$

$\langle \text{variable_assign} \rangle ::= \langle \text{identifier} \rangle \text{"="} \langle \text{command_statement} \rangle \text{";"}$

$\langle \text{function_def} \rangle ::= \text{"function"} \langle \text{identifier} \rangle$
 $\quad \quad \quad \text{"("} \langle \text{opt_param_list} \rangle \text{")"} \text{" ":"}$
 $\quad \quad \quad \langle \text{statement_block} \rangle$

$\langle \text{opt_param_list} \rangle ::= \langle \text{param_list} \rangle$
 $\quad \quad \quad | \langle \text{empty} \rangle$

`<param_list> ::= <identifier> <param_list_tail>`

`<param_list_tail> ::= "," <identifier> <param_list_tail>
| <empty>`

`<statement_block> ::= <statement> <statement_block>
| <empty>`

`<command_statement> ::= <scan_command>
| <filter_command>
| <sort_command>
| <limit_command>
| <summary_command>
| <find_duplicates_command>
| <merge_command>
| <output_command>
| <new_table_command>
| <function_call>`

`<scan_command> ::= "scan" <table_column> "using" "tool" <tool_name>
>`

`<tool_name> ::= <string>`

`<table_column> ::= <table_name> "." <column_name>`

`<table_name> ::= <string>`

<column_name> ::= <string>

<filter_command> ::= "filter" <identifier> "where" <condition>

<condition> ::= <or_condition>

<or_condition> ::= <and_condition> "OR" <or_condition>
| <and_condition>

<and_condition> ::= <expression> "AND" <and_condition>
| <expression>

<expression> ::= <identifier> <comparison_op> <value>
| <identifier> <string_match_op> <value>

<comparison_op> ::= ">"
| "<"
| ">="
| "<="
| "=="
| "!="

<string_match_op> ::= "starts_with"
| "contains"
| "ends_with"

<sort_command> ::= "sort" <identifier>

["by"] <identifier> <opt_sort_direction>

<opt_sort_direction> ::= "asc"
 | "desc"
 | <empty>

<limit_command> ::= "limit" <identifier> <number>

<summary_command> ::= "summarize" <identifier> <summary_tail>

<summary_tail> ::= ", " <identifier> <summary_tail>
 | <empty>

<find_duplicates_command> ::= "findLocalDuplicates" <identifier>
 "columns" "[" <column_list> "]"
 <opt_using_tool>

<opt_using_tool> ::= "using" "tool" <string>
 | <empty>

<column_list> ::= <string> <column_list_tail>

<column_list_tail> ::= ", " <string> <column_list_tail>
 | <empty>

<merge_command> ::= "merge" <identifier>

"->" "Here" <opt_merge_offset> <
opt_allow_danger>

<opt_merge_offset> ::= "(" <number> ")"
| <empty>

<opt_allow_danger> ::= "allowDanger"
| <empty>

<output_command> ::= "output" <identifier> "->" <output_target>

<output_target> ::= "console"
| "file" <string>
| "Here" <opt_output_offset>

<opt_output_offset> ::= "(" <number> ")"
| <empty>

<new_table_command> ::= "newTable" <string>
<opt_from>
<opt_add_columns>

<opt_from> ::= "from" <identifier>
| <empty>

`<opt_add_columns> ::= "addColumnns" "[" <column_func_list> "]"`
`| <empty>`

`<column_func_list> ::= <column_func> <column_func_list_tail>`

`<column_func_list_tail> ::= ", " <column_func> <column_func_list_tail>`
`| <empty>`

`<column_func> ::= <string>`
`| <func_call>`

`<func_call> ::= <identifier> "(" <opt_arg_list> ")"`

`<function_call> ::= <identifier> "(" <opt_arg_list> ")"`

`<opt_arg_list> ::= <arg_list>`
`| <empty>`

`<arg_list> ::= <value> <arg_list_tail>`

`<arg_list_tail> ::= ", " <value> <arg_list_tail>`
`| <empty>`

`<value> ::= <identifier>`
`| <string>`
`| <number>`

`<identifier> ::= <letter> <identifier_tail>`

`<identifier_tail> ::= <letter> <identifier_tail>
| <digit> <identifier_tail>
| "_" <identifier_tail>
| <empty>`

`<number> ::= <digit> <number_tail>`

`<number_tail> ::= <digit> <number_tail>
| <empty>`

`<string> ::= ''' <string_body> '''`

`<string_body> ::= <any_char_except_quote> <string_body>
| <empty>`

(* Note: `<opt-spaces>` and `<OneOrMoreSpaces>` may appear between any two tokens (except inside literals, identifiers, numbers and multi-character operators). They are shown only in the top-level rules for brevity; the rest of the grammar follows the same whitespace-insensitive convention. *)

2.2.4 Core Syntactic Structures

Program and Statements

A DaQuVa program is a flat sequence of statements with no required entry-point. Execution proceeds top-to-bottom. There are four statement kinds:

- **Database initialisation** (`my_db`): establishes the active connection;

- **Variable assignment:** binds the result of a command to a name;
- **Function definition:** introduces a named, reusable routine;
- **Command statement:** executes an action without binding.

Expressions and Conditions

Conditions used in `filter` commands consist of one or more *expressions* joined by AND / OR. Each expression either compares a column-level attribute (e.g. `dirty_level`) to a value using a standard comparison operator, or applies one of the supported string predicates. This deliberately limited expression language prevents arbitrary logic and keeps programs auditable.

Functions

Functions accept a comma-separated list of positional parameters and contain an indentation-delimited block of statements. Unlike general-purpose languages, DaQuVa functions may only contain the same statement types available at the top level; there is no nested function definition or recursion.

2.3 Basic DSL Semantics

2.3.1 Execution Model

DaQuVa programs execute *sequentially*. Each statement is fully evaluated before the next begins. The runtime maintains:

- A single **database context** set by `my_db`;
- A **variable environment** mapping names to result sets;
- A **function table** mapping names to parameter lists and statement blocks.

There is no implicit concurrency, no exception-handling syntax, and no dynamic typing — every variable holds a *result set* (a typed, annotated table-like structure).

2.3.2 Semantic Description of Core Constructs

Table 2.3.1 summarises the principal semantic entities of DaQuVa.

Table 2.3.1 - Core semantic entities of DaQuVa

Entity	Description
my_db	Declares the active database connection using an opaque connection string. Exactly one my_db declaration is expected per program; re-declaration overwrites the current context.
Table	The primary data container. Tables exist either in the connected database or as in-memory result sets produced by scan or newTable.
Variable	Any command result can be bound to a variable. Variables are untyped at the syntax level but carry a <i>result-set</i> value at runtime.
Tool	An external validator or AI-powered analyser identified by a string name (e.g. "fuzzy_matcher", "typo_checker"). The runtime resolves the name to a registered tool implementation.
Column / Meta-variable	Columns such as dirty_level and duplicate_score are injected into result sets automatically by scan and findLocalDuplicates, respectively. They may also be added explicitly via addColumns.
Function	A named, reusable sequence of commands parameterised by identifiers. Calling a function executes its body in a new scope; parameters shadow outer variables.
Threshold / Confidence	An integer in the range [0,100] representing a minimum quality score. Used in filter conditions and as the confidence argument to Here().
Here()	A special sink that redirects writes into the live database. Requires an optional confidence argument; when used with merge, it writes merged records back to the source.
allowDanger	A boolean flag that must be present on any merge command that targets Here(). It serves as an explicit acknowledgment that the operation modifies persisted data.

2.3.3 Command Semantics

scan — Column Validation

`<scan_command> ::= "scan" <table_column> "using" "tool" <string>`

Meaning. scan passes every value in the named column of the named table to the specified tool. The tool assigns each row a dirty_level score in [0,100], where 100 represents maximum dirtiness. The result is an annotated copy of the rows with the added meta-column.

Execution.

- Resolve the table name against the active database context;

- Resolve the tool name against the registered tool registry;
- Invoke the tool for each row value in the column;
- Return a result set containing all original columns plus `dirty_level`.

Errors. An error is raised if the table or column does not exist, if the tool name cannot be resolved, or if the database context has not been initialised.

`filter` — Boolean and String Filtering

`<filter_command> ::= "filter" <identifier> "where" <condition>`

Meaning. Produces a subset of the input result set containing only rows that satisfy the given condition.

Execution. Each row is evaluated against the condition; rows for which the condition is true are retained. Conditions may combine multiple predicates with AND / OR; AND binds more tightly than OR, matching standard boolean precedence. Predicates may compare numeric or string values with `==`, `!=`, `>`, `<`, `>=`, and `<=`, or apply string tests with `starts_with`, `contains`, and `ends_with`.

```
paid_adults = filter raw where age >= 18 AND plan != "free";
major_providers = filter raw where email contains "@gmail" OR email
    contains "@outlook";
org_users = filter raw where email ends_with ".org";
```

Errors. An error is raised if the referenced variable is undefined, if a column referenced in a condition does not exist in the result set, or if operand types are incompatible.

`sort` and `limit` — Result-Set Ordering

`<sort_command> ::= "sort" <identifier> [ "by" ] <identifier> [ "asc" | "desc" ]`

`<limit_command> ::= "limit" <identifier> <number>`

Meaning. `sort` produces a table ordered by a selected column, while `limit` returns only the first n rows. Together they support common exploration workflows such as inspecting the oldest customers, highest scores, or first few suspicious records.

```
top5 = sort raw by age desc;
top5 = limit top5 5;
```

Execution. In-memory tables are ordered by a stable mixed-type sort key so that empty, numeric, boolean, and string values can be handled without runtime type errors. When the source plan is backed by SQLite, sorting and limiting are pushed down into ORDER BY and LIMIT clauses.

summarize — Aggregate Proof Summary

```
<summary_command> ::= "summarize" <identifier> { ",", <identifier> }
```

Meaning. `summarize` takes one or more previously bound result sets (the *stages* of a pipeline) and produces a single new table in which *each named stage becomes one row of aggregate metrics*. Unlike every other command, it does not add columns to the data — it reduces each stage to a fixed schema of counts, giving an inspectable proof that a transformation actually happened.

Result schema. The produced table has a fixed set of columns: `stage`, `row_count`, `column_count`, `duplicate_group_count`, `duplicate_candidate_count`, `typo_suspect_count`, `input_rows_represented`, and `rows_removed_by_merge`. The last two are derived from the `merged_from_count` annotation so that a merge which collapses several input rows into one is reported as rows represented versus rows removed.

```
proof = summarize raw , paid_adults , dupes , merged ;
proof -> file "output/proof_summary.csv";
save proof into proof_summary allowDanger;
```

Execution. For each stage, the engine materialises the underlying plan and computes the metrics in a single pass over the rows; duplicate groups are counted from distinct `duplicate_group_id` values, typo suspects from a truthy `typo_suspect` flag, and represented input rows from the merge count. The resulting summary is an ordinary result set, so it can be exported to a CSV file or persisted to SQLite like any other table.

Errors. An error is raised if any referenced stage variable is undefined, using the same `UndefinedVariable` condition as the other commands.

findLocalDuplicates — Duplicate Detection

```
<find_duplicates_command>
::= "findLocalDuplicates" <identifier>
    "columns" "[" <column_list> "]"
    [ "using" "tool" <string> ]
```

Meaning. Identifies rows within the input result set that are potential duplicates according to the specified key columns. An optional AI-powered tool (e.g. a fuzzy matcher) provides similarity scoring. The implementation accepts both `fuzzy_duplicates` and `levenshtein_matcher`; the latter is an explicit alias for the same Levenshtein-distance based duplicate detection strategy.

Execution. Rows are grouped by the specified columns; within each group, pairs are scored for similarity. The result set contains all candidate duplicate rows annotated with a `duplicate_score` in `[0,100]`.

Errors. An error is raised if the input variable is undefined or if any listed column does not exist in the result set.

`merge` — **Duplicate Resolution**

```
<merge_command> ::= "merge" <identifier>
                    "->" "Here" [ "(" <number> ")" ]
                    [ "allowDanger" ]
```

Meaning. Merges duplicate records in the input result set and writes the resolved records back to the live database through `Here()`. The optional numeric argument specifies a minimum confidence threshold; only duplicate pairs with `duplicate_score` $\geq$ threshold are merged.

Execution.

- Filter candidates by the confidence threshold (default: 0);
- For each qualifying pair, apply a deterministic merge strategy (e.g. keep the most complete record);
- Write merged records to the database; delete superseded records.

Safety constraint. The `allowDanger` flag *must* be present. Its absence is a compile-time error. This design makes destructive database modifications opt-in and visible in code review.

`output` — **Result Export**

```
<output_command> ::= "output" <identifier> "->" <output_target>
<output_target>  ::= "console" | "file" <string> | "Here" [ "(" <number>
    > ")" ]
```

Meaning. Writes a result set to one of three sinks: the console (standard output), a named file, or the live database via `Here()`.

Execution.

- `console` — serialises rows as human-readable text;
- `file "<path>"` — writes rows to the specified file path in a delimited format (e.g. CSV);
- `Here()` — inserts or upserts rows into the database.

Errors. An error is raised if the variable is undefined, if the file path is not writable, or if the database context is missing for `Here()`.

`newTable` — **Table Construction**

```
<new_table_command> ::= "newTable" <string>
                      [ "from" <identifier> ]
                      [ "addColumnns" "[" <column_func_list> "]" ]
```

Meaning. Creates a new named table, optionally seeded from an existing result set and optionally extended with computed columns.

Execution. If `from` is provided, the new table is initialised with the rows of the referenced result set. Each entry in `addColumnns` is either a plain column name (copied from the source) or a function call whose result becomes a new column value (e.g. `length("name")`).

function and Function Calls

```
<function_def> ::= "function" <identifier> "(" [ <param_list> ] ")"
                ":"
```

```
                <statement_block>
```

```
<function_call> ::= <identifier> "(" [ <arg_list> ] ")"
```

Meaning. Functions encapsulate reusable command sequences. They are *first-defined* (must be defined before use in a top-to-bottom interpreter) and execute in their own variable scope. Parameters are positional; the number of arguments at the call site must match the parameter list. Function bodies may return either a value/variable or the direct result of another function call, which allows small helper functions to delegate to more specialized routines.

Errors. Arity mismatch, calling an undefined function, or a return statement referencing an undefined variable all raise errors.

2.3.4 Error Catalogue

Table 2.3.2 catalogues the principal errors and their triggers.

Table 2.3.2 - Error catalogue for DaQuVa.

Error	Condition
UndefinedVariable	An identifier is used before assignment.
UndefinedTable	A table name cannot be resolved in the database context.
UndefinedColumn	A column referenced in a condition or column list does not exist in the result set.
UnresolvedTool	A tool string cannot be matched to a registered tool.
ArityMismatch	A function call provides more or fewer arguments than parameters declared.
MissingContext	A database-dependent command is issued before <code>my_db</code> .
MissingAllowDanger	A <code>merge -> Here()</code> command lacks the <code>allowDanger</code> flag.
InvalidThreshold	A numeric argument to <code>Here()</code> or a filter expression falls outside <code>[0, 100]</code> .
IOError	The file path in an <code>output -> file</code> command is not accessible.

2.4 Illustrative Example Program

The program below demonstrates all major language constructs. It connects to a database, defines a reusable email-cleaning function, applies it, exports results, detects and merges duplicates, and creates a summary table for further analysis.

Listing 2.4.1 - Annotated DaQuVa example program.

```
# === 1. Database connection ===
my_db "some_connection_string_goes_here";

# === 2. Reusable function: clean emails in any table ===
function cleanEmails(table_name, threshold):
    # Scan the email column with an AI validator
    potential_dirty = scan table_name."email" using tool "
        misa_ai_validator";

    # Keep only rows whose dirty_level exceeds the threshold
    high_risk = filter potential_dirty where dirty_level > threshold;

    return high_risk;

# === 3. Apply the function to the "users" table ===
high_risk_emails = cleanEmails("users", 70);

# === 4. Export results ===
output high_risk_emails -> file "high_risk_emails.csv";
output high_risk_emails -> console;

# === 5. Explore high-risk records with string filters and top-N
    ordering ===
major_providers = filter high_risk_emails
    where email contains "@gmail" OR email contains "
        @outlook";
top_risk = sort high_risk_emails by dirty_level desc;
top_risk = limit top_risk 5;
output major_providers -> console;
```

```

output top_risk -> console;

# === 6. Detect near-duplicate records among high-risk rows ===
duplicates = findLocalDuplicates high_risk_emails
           columns ["name", "dob"]
           using tool "levenshtein_matcher";

# === 7. Merge duplicates back into the database (confidence >= 90%)
===
merge duplicates -> Here(90) allowDanger;

# === 8. Create a summary table for downstream analysis ===
analysis_table = newTable "high_risk_analysis"
                 from high_risk_emails
                 addColumns [duplicate_score, length("name")];

# === 9. Additional validation pass on the summary table ===
cli_result = scan analysis_table."email" using tool "typo_checker";
output cli_result -> console;

```

The program illustrates several semantic properties:

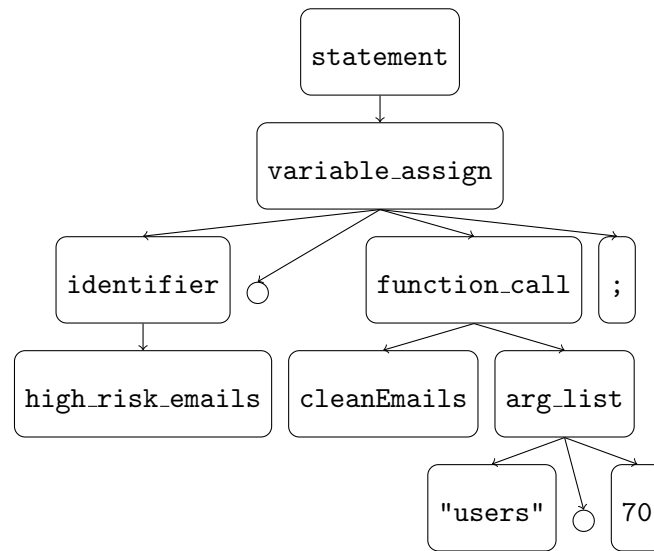
- The function `cleanEmails` abstracts a two-step scan-then-filter pipeline, making the threshold a tune-able parameter.
- The output command is used twice on the same variable, demonstrating that result sets are non-destructively consumed.
- `contains`, `sort`, and `limit` support interactive exploration without leaving the DSL.
- The merge command requires `allowDanger` because it writes to the live database, making the risky operation immediately visible to reviewers.
- `newTable` constructs a derived table that mixes copied data with computed columns (`duplicate_score`, `length("name")`), enabling further analysis without modifying the source table.

2.5 Derivation Trees for Control Structures

Below are the derivation trees for the main control structures of DaQuVa.

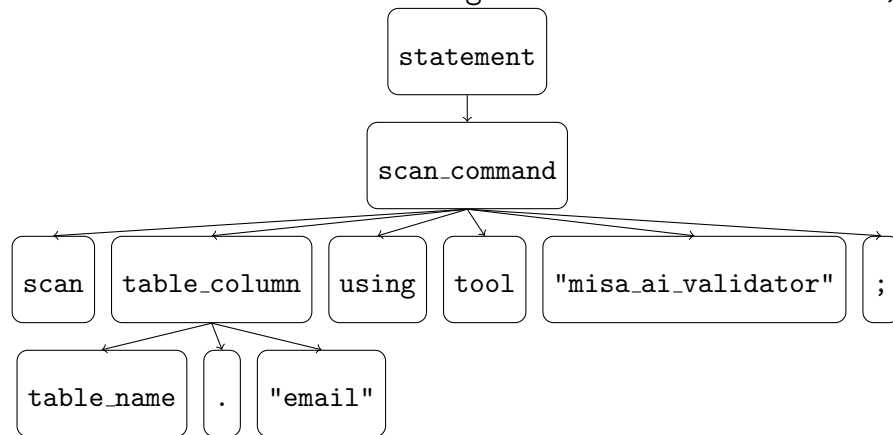
2.5.1 Variable Definition

Example: `high_risk_emails = cleanEmails("users", 70);`



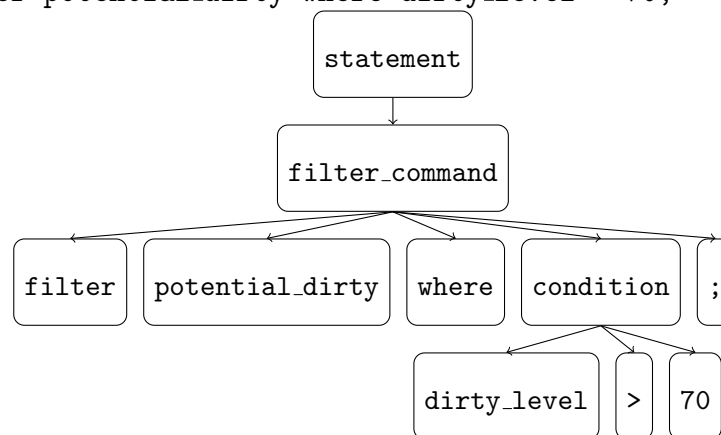
2.5.2 scan Command

Example: scan table_name."email" using tool "misa_ai_validator";



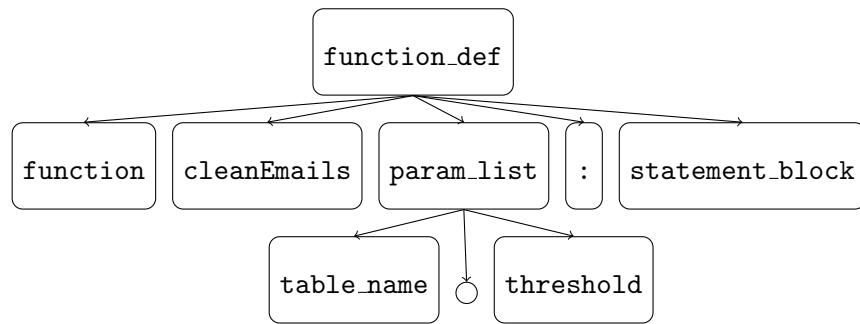
2.5.3 filter Command

Example: filter potential_dirty where dirty_level > 70;



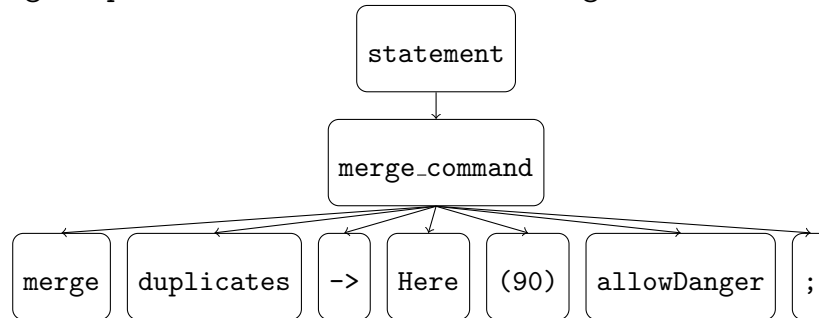
2.5.4 Function Definition

Example: function cleanEmails(table_name, threshold):



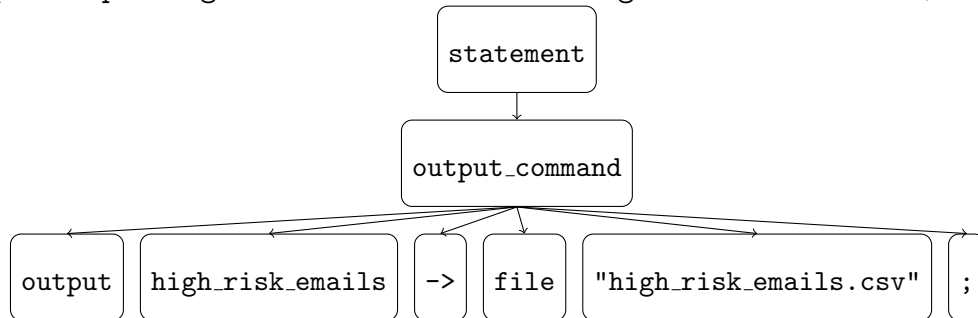
2.5.5 merge Command

Example: `merge duplicates -> Here(90) allowDanger;`



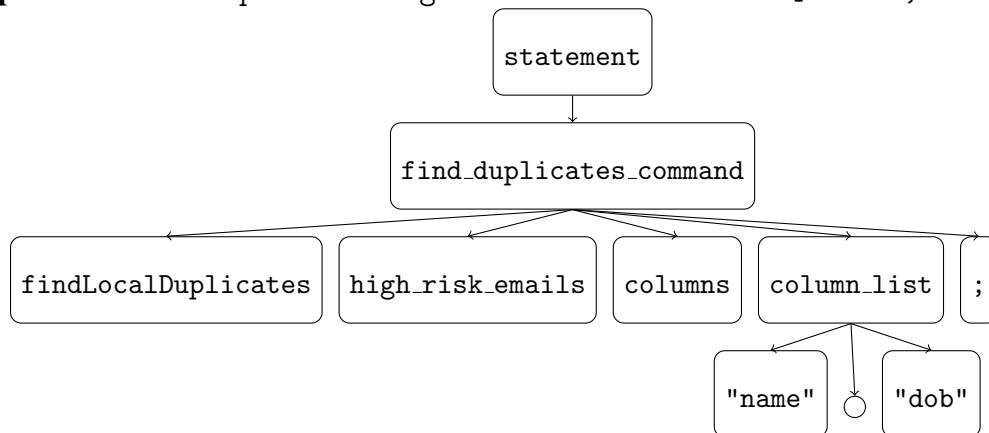
2.5.6 output Command

Example: `output high_risk_emails -> file "high_risk_emails.csv";`



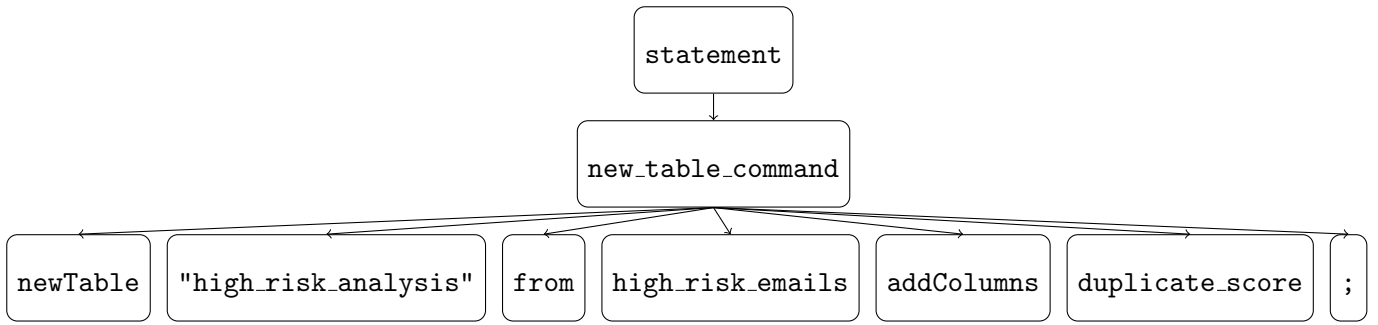
2.5.7 findLocalDuplicates Command

Example: `findLocalDuplicates high_risk_emails columns ["name","dob"];`



2.5.8 newTable Command

Example: `newTable "high_risk_analysis" from high_risk_emails addColumns [duplicate.s`



2.6 Summary

This report has presented the complete formal grammar and semantic specification of the **Data Quality Validator** DSL. The BNF grammar in Section 2.2 defines an unambiguous, minimal syntax covering database initialisation, column scanning, result filtering, duplicate detection and merging, result ordering, aggregate proof summaries, result export, and table construction. The semantic specification in Section 2.3 assigns a precise meaning to each construct, documents execution rules, and enumerates the error conditions the runtime must detect and report.

The language’s design deliberately constrains expressiveness in favour of safety and auditability: destructive operations require explicit opt-in flags, and the expression language is limited to comparisons, boolean composition, and auditable string predicates rather than arbitrary side-effecting code. These properties make DaQuVa programs straightforward to review, test, and extend.

3 DSL Implementation

The implementation of the DaQuVa DSL follows a classic compiler pipeline: lexer $\rightarrow$ parser $\rightarrow$ AST. All components are written in pure Python 3 and are directly derived from the formal BNF grammar presented in Chapter 4. The code is kept minimal, readable, and fully aligned with the safety-first and readability principles of the language.

3.1 Lexer

The lexer is a hand-written, regex-based tokenizer that converts the raw source code into a flat stream of tokens. It explicitly recognises every terminal symbol defined in the BNF (keywords, identifiers, literals, operators, punctuation) while discarding insignificant whitespace and comments. Although the grammar now defines `SingleSpace` and `OneOrMoreSpaces` for formal completeness, the lexer follows the standard convention (explicitly stated in the BNF note) and treats whitespace purely as a token separator.

Key design decisions:

- All reserved words are turned into a single KW token;
- Whitespace (`[ \t\r\n]+`) and single-line comments (`#.*`) are silently skipped;
- Multi-character operators (`->`, `==`, `!=`, `>=`, `<=`) are recognised before single-character ones;
- String literals have their surrounding quotes stripped;
- Numbers are immediately converted to Python `int`.

Listing 3.1.1 - Core lexer implementation (excerpt)

```
KEYWORDS = frozenset({
    'my', 'db', 'function', 'return', 'end', 'scan', 'filter', 'merge',
    'output', 'newTable', 'findLocalDuplicates', 'where', 'AND', 'OR',
    'starts_with', 'contains', 'ends_with', 'sort', 'by', 'asc', 'desc',
    'limit', 'summarize',
    'columns', 'using', 'tool', 'console', 'file', 'from', 'addColumnns',
    'Here', 'allowDanger'
})

class Lexer:
    _RULES = [
        ('COMMENT', r'#[^\n]*'),
        ('STRING', r'"[^"]*"')
```

```
( 'NUMBER' ,      r '\d+' ),
( 'ARROW' ,       r '->' ),
( 'COMP_OP' ,     r '!=|==|>|=|<|=|>|<') ,
( 'COLON' ,       r ':' ),
( 'ASSIGN' ,      r '=' ),
( 'SEMICOLON' ,   r ';' ),
( 'LPAREN' ,      r '\(' ),
( 'RPAREN' ,      r '\)' ),
( 'LBRACKET' ,    r '[' ),
( 'RBRACKET' ,    r ']' ),
( 'DOT' ,         r '\.' ),
( 'COMMA' ,       r ',' ),
( 'WORD' ,        r '[A-Za-z_]\w*' ),
( 'SKIP' ,        r '[ \t\r\n]+' ),
( 'MISMATCH' ,    r '.' ),
```

```
]

```

```
def __init__(self, source: str):
    self.tokens = []
    self.pos = 0
    for mo in re.finditer('|'.join(f'(?P<{n}>{p})' for n,p in self.
        _RULES), source):
        kind = mo.lastgroup
        if kind in ('SKIP', 'COMMENT'):
            continue
        if kind == 'MISMATCH':
            raise LexerError(...)
        if kind == 'STRING':
            self.tokens.append(Token('STRING', mo.group()[1:-1], mo
                .start()))
        elif kind == 'NUMBER':
            self.tokens.append(Token('NUMBER', int(mo.group()), mo.
                start()))
```

```

elif kind == 'WORD':
    k = 'KW' if mo.group() in KEYWORDS else 'ID'
    self.tokens.append(Token(k, mo.group(), mo.start()))
else:
    self.tokens.append(Token(kind, mo.group(), mo.start()))

def peek(self, offset=0):
    idx = self.pos + offset
    return self.tokens[idx] if idx < len(self.tokens) else Token('
    EOF', None, -1)

def consume(self, kind=None, value=None):
    tok = self.peek()
    if kind and tok.kind != kind or value and tok.value != value:
        raise SyntaxError(...)
    self.pos += 1
    return tok

```

The lexer produces exactly the token stream expected by the recursive-descent parser and handles all edge cases from the illustrative example in Section 4.4.

3.2 Parser

The parser is a classic recursive-descent implementation that mirrors the BNF structure one-to-one. It uses the lexer's peek/consume interface and builds the AST on the fly. Two small practical adaptations were introduced for real-world usability while staying faithful to the formal grammar:

- Function bodies are terminated by the keyword `end` (the original BNF used pure recursion without an explicit closer; `end` makes the language non-indentation-sensitive);
- The `return` statement (already present in the keyword table) was given its own AST node.

Listing 3.2.1 - Parser entry point and statement dispatcher (excerpt)

```

class Parser:
    def parse(self) -> Program:
        stmts = []
        while not self._at_end():
            stmts.append(self._parse_statement())
        return Program(stmts)

```

```

def _parse_statement(self):
    tok = self._peek()
    if tok.kind == 'KW':
        if tok.value == 'my':            return self._parse_db_init()
        if tok.value == 'function':      return self._
            _parse_function_def()
        if tok.value == 'return':        return self._parse_return()
    node = self._parse_command()
    self._consume('SEMICOLON')
    return node
    if tok.kind == 'ID':
        if self._is('ASSIGN', offset=1):
            return self._parse_assignment()
        if self._is('LPAREN', offset=1):
            node = self._parse_function_call()
            self._consume('SEMICOLON')
            return node
    raise ParseError(...)

```

Individual command parsers (scan, filter, findLocalDuplicates, merge, # output, newTable) follow the BNF productions exactly and are omitted # here for brevity. Each returns the corresponding AST node.

Every parsing method directly corresponds to a BNF production (e.g. `_parse_scan` implements `<scan_command>`). *Compound filters are parsed through separate `_or_condition` and `_and_condition` methods,*

3.3 AST

The Abstract Syntax Tree is defined with Python dataclasses, giving a clean, immutable and type-safe representation that exactly mirrors the BNF non-terminals. All nodes inherit from a common `ASTNode` base class.

Listing 3.3.1 - Complete AST node hierarchy

```
@dataclass
```

```

class ASTNode: pass

@dataclass
class Program(ASTNode):
    statements: List[ASTNode]

@dataclass
class DatabaseInit(ASTNode):
    connection: str

@dataclass
class FunctionDef(ASTNode):
    name: str
    params: List[str]
    body: List[ASTNode]

@dataclass
class Assignment(ASTNode):
    name: str
    value: ASTNode

@dataclass
class ReturnStatement(ASTNode):
    value: ASTNode

@dataclass
class Scan(ASTNode):
    table: ASTNode          # Literal | Variable
    column: str
    tool: str

@dataclass
class Filter(ASTNode):

```



```

        source: str
        condition: ASTNode

@dataclass
class CompoundCondition(ASTNode):
    left: ASTNode
    op: str
    right: ASTNode

@dataclass
class FindDuplicates(ASTNode):
    source: str
    columns: List[str]
    tool: Optional[str]

@dataclass
class Merge(ASTNode):
    source: str
    threshold: Optional[int]

@dataclass
class Output(ASTNode):
    source: str
    target: str
    path: Optional[str] = None
    threshold: Optional[int] = None

@dataclass
class NewTable(ASTNode):
    name: str
    source: Optional[str]
    columns: List[ASTNode]

```

```

@dataclass
class SortExpression(ASTNode):
    source: str
    column: str
    ascending: bool

@dataclass
class LimitExpression(ASTNode):
    source: str
    count: int

@dataclass
class SummaryExpression(ASTNode):
    source_names: Tuple[str, ...]    # one or more stages to summarize

@dataclass
class BinaryOp(ASTNode):
    left: ASTNode
    op: str
    right: ASTNode

@dataclass
class FunctionCall(ASTNode):
    name: str
    args: List[ASTNode]

@dataclass
class Variable(ASTNode):
    name: str

@dataclass
class Literal(ASTNode):
    value: Union[int, str]

```

A simple `print_ast` pretty-printer (included in the implementation) produces human-readable tree views for debugging. The AST serves as the single source of truth for all subsequent phases (semantic analysis, execution engine, pretty-printing, etc.).

The lexer, parser, and AST together form a complete front-end that can already parse every valid DaQuVa program defined in the language specification.

3.4 Semantic Analysis and Runtime

The runtime phase resolves parsed AST nodes against the active execution context. It registers CSV and SQLite connections, stores in-memory variables, tracks user-defined functions, and turns table expressions into lazy logical plans. This design keeps parsing independent from file systems and databases: the parser only validates syntax, while the runtime validates names, columns, tool availability, and output destinations.

Important runtime checks added in the current implementation include:

- **Column validation:** filter conditions and projected columns are checked against the available schema. A misspelled condition column now raises an explicit error that lists available columns instead of silently producing an empty result;
- **Function return handling:** function bodies may return a variable, literal, or another function call, enabling composition of small reusable routines;
- **Controlled persistence:** output to a database destination accepts an `allowDanger` flag. The SQLite writer uses this flag when replacing persisted data, keeping potentially destructive writes visible in DSL scripts;
- **Tool aliases:** `levenshtein_matcher` is resolved as an alias of the fuzzy duplicate detection implementation.

3.5 Execution Engine

The execution engine evaluates lazy logical plans and chooses the most suitable backend strategy. For CSV-backed data, plans are materialized and evaluated in Python. For SQLite-backed data, simple relational operations are compiled into SQL whenever possible.

The latest implementation extends the executable plan set with `SortPlan` and `LimitPlan`. In Python execution, `sort` uses a mixed-type safe key so that empty values, numbers, booleans, and strings do not produce unhandled `TypeError` exceptions. For SQLite plans, `filter`, `sort`, and `limit` are pushed down into SQL: `WHERE`, `ORDER BY`, and `LIMIT` are generated directly against the source table.

String predicates are also executable in both backends. In memory, they are evaluated with Python string operations. In SQLite, `starts_with`, `contains`, and `ends_with` are translated into parameterized `LIKE` expressions:

```
email starts_with "a"      -> email LIKE "a%"
email contains "@gmail"    -> email LIKE "%@gmail%"
email ends_with ".org"     -> email LIKE "%.org"
```

Several correctness fixes were made in the execution layer. CSV input values are stripped of surrounding whitespace before storage, preventing equality filters from failing on visually identical strings. Numeric comparisons now produce a clear `ValueError` when ordered comparisons are attempted against incompatible non-numeric strings. SQLite table replacement was also made safer by creating a temporary replacement table and atomically renaming it only after creation succeeds.

The plan set is further extended with a `SummaryPlan`, which carries a tuple of `SummarySource` entries (the stage name, its logical plan, and its column list). When the engine encounters a `SummaryPlan` it materialises each source and reduces it to one metrics row via a small `_summary_row` helper. That helper counts distinct `duplicate_group_id` values, sums the `merged_from_count` annotation to recover how many input rows a merged result represents, and uses a tolerant truthiness check for the `typo_suspect` flag so that string, boolean, and numeric encodings are all interpreted consistently. Because the summary is emitted as an ordinary list of row dictionaries with a fixed `SUMMARY_COLUMNS` schema, it flows through the existing CSV and SQLite writers unchanged.

3.6 Interactive REPL

DaQuVa now includes an interactive REPL started with:

```
python -m daquva --repl
```

The REPL keeps a persistent runtime context, so users can declare connections, assign intermediate tables, inspect variables, and print results without rerunning an entire script. It buffers multi-line input until a semicolon or function-closing brace is present, then parses and executes the accumulated snippet. Meta-commands provide lightweight inspection:

- `:help` prints available REPL commands and examples;
- `:tables` lists active connections;
- `:vars` lists in-memory variables and table schemas;
- `:quit` exits the session.

After an assignment, the REPL reports the variable name, row count, column count, and column names. This immediate feedback is useful for a demonstration setting because each transformation can be inspected before the next command is entered.

3.7 Showcase Demo

A new end-to-end showcase script was added under `code/demo/demo_showcase`. It demonstrates the latest implemented features in one narrative pipeline: typo detection, compound AND/OR filters, provider

filtering with `contains`, top-*n* exploration with `sort` and `limit`, domain filtering with `ends_with`, fuzzy duplicate detection and merge, CSV export, and SQLite persistence with `allowDanger`.

To make the pipeline auditable, the demo input was enlarged to 30 customer rows containing deliberate near-duplicates and email typos, and the script ends with a `summarize` step that proves the cleaning effect:

```
proof = summarize raw , paid_adults , dupes , merged ;  
proof -> file "output/proof_summary.csv";  
save proof into proof_summary allowDanger;
```

This closes the loop required of a data-quality tool: a concrete CSV *input*, cleaned CSV and SQLite *outputs*, and an aggregate summary that is itself written to both CSV (`output/proof_summary.csv`) and a SQLite table (`proof_summary`). The summary does not merely append columns; it reports before/after row counts so a reviewer can confirm, for example, that 30 raw rows were reduced to 15 merged rows representing 25 inputs.

4 Evaluation & Experiments

4.1 Experiment 1: Core Data Quality Pipeline

The main evaluation strategy for DaQuVa is example-driven: each demo script executes a focused data-quality workflow and writes observable outputs as CSV or SQLite artifacts. This is appropriate for the prototype stage because the language is small, the expected behavior is table-oriented, and the produced tables can be inspected directly.

The existing demo set validates the main pipeline operations:

- scanning input tables and annotating rows with tool-generated metadata;
- filtering by numeric thresholds and string values;
- detecting potential duplicates and merging duplicate groups;
- exporting cleaned data to CSV files and SQLite databases;
- composing several commands into a longer processing pipeline.

4.2 Experiment 2: Latest Language Features

The last implementation iteration adds a dedicated showcase demo in `code/demo/demo_showcase/showcase_demo.sql`. This script exercises the features added after the initial prototype and acts as a compact five-minute demonstration scenario.

```
paid_adults = filter raw where age >= 18 AND plan != "free";
major_providers = filter raw where email contains "@gmail" OR email
    contains "@outlook";
top5 = sort raw by age desc;
top5 = limit top5 5;
org_users = filter raw where email ends_with ".org";
dups = find_duplicates adult_paid[name] using tool fuzzy_duplicates(2)
    ;
merged = merge dups;
save merged into clean_customers allowDanger;
```

This experiment validates that compound boolean filters respect precedence, string predicates work on realistic email data, ordering and limiting compose as expected, duplicate analysis still works after filtering, and the final cleaned result can be persisted to SQLite only when the script explicitly contains `allowDanger`.

4.3 Experiment 3: Robustness and Correctness Checks

Several implementation fixes improve the reliability of the evaluator:

- CSV string cells are normalized by stripping surrounding whitespace, which makes equality filters behave consistently;
- Filters that reference unknown columns now raise an explicit error with the available column names;
- Ordered comparisons against incompatible non-numeric values now produce a clear `ValueError` instead of an unhandled Python exception;
- The Levenshtein duplicate matcher short-circuits row pairs whose length difference already exceeds the configured distance threshold;
- SQLite replacement writes use a temporary table and atomic rename pattern, reducing the risk of losing existing data if table creation fails.

Together, these checks make the prototype more suitable for iterative use: invalid scripts fail loudly, common data-format issues are handled before comparison, and persistence operations are more conservative.

4.4 Experiment 4: Interactive Use

The REPL was added to evaluate incremental authoring. Running `python -m daquva --repl` starts a persistent session where users can declare connections, execute one statement at a time, inspect variables with `:vars`, inspect active connections with `:tables`, and print intermediate tables. The REPL reports row and column counts after assignments, which makes it easier to validate each transformation before continuing the pipeline.

4.5 Experiment 5: Input/Output Proof Summary

A data-quality tool should make it easy to prove that processing happened: it needs a concrete input, observable outputs, and an output that is a *useful summary* rather than just more annotated columns. The showcase demo satisfies this end to end. It reads a 30-row input CSV (`demo_showcase/input/customers.csv`), writes cleaned data to both a CSV file (`output/cleaned_customers.csv`) and a SQLite table (`clean_customers`), and finishes with the `summarize` command, whose result is exported to `output/proof_summary.csv` and persisted to the SQLite table `proof_summary`.

Running the demo produces the proof table in Table 4.5.1, which collapses each pipeline stage to a single row of metrics. It shows directly that filtering removed five rows (30 raw to 25 paid adults), that duplicate analysis flagged ten groups across twenty candidate rows, and that merging reduced the 25 candidates to 15 rows while still representing all 25 inputs — i.e. ten rows were removed by merge.

Table 4.5.1 - Aggregate proof summary produced by the showcase demo.

stage	rows	dup. groups	dup. cand.	inputs repr.	removed
raw	30	0	0	30	0
paid_adults	25	0	0	25	0
dupes	25	10	20	25	0
merged	15	0	0	25	10

Because the summary is itself a result set, it is verifiable two ways: by opening the exported CSV, or by querying the `proof_summary` table in the SQLite database alongside the cleaned `clean_customers` table. This makes the input-to-output transformation auditable without re-running the pipeline.

5 Discussion

5.1 Strengths of the Current Prototype

The strongest aspect of the current DaQuVa prototype is that the language now covers a complete cleaning workflow rather than isolated syntax examples. A user can load CSV or SQLite data, run scans, combine filters with AND/OR, apply string predicates, inspect the most relevant rows with `sort` and `limit`, detect near duplicates, merge them, and export the cleaned table. The `summarize` command rounds out this workflow with an aggregate proof table that reports before/after row counts for each stage, so a reviewer can confirm that cleaning actually reduced the data rather than just inspecting individual rows. The new REPL strengthens the workflow further because it supports incremental exploration without repeatedly running a full script.

The implementation also keeps destructive operations visible. Database writes require explicit syntax such as `allowDanger`, and the SQLite output path uses a safer temporary-table replacement strategy. This matches the original safety-first design goal of the language.

5.2 Limitations and Future Work

The prototype is still intentionally small. The expression language supports comparisons, boolean composition, and a few string predicates, but it does not yet provide joins, date operations, or user-defined validators. Aggregation is currently limited to the fixed-schema `summarize` proof table; there is no general `group by` with user-chosen aggregate functions. The REPL is useful for exploration, but it does not yet provide autocomplete, command history persistence, or static diagnostics before execution.

Future work should focus on broadening the type system, adding a formal test suite around the demo scenarios, and extending SQL pushdown to more operations. Another important direction is improving documentation synchronization so that the BNF, parser implementation, examples, and report remain aligned as the DSL evolves.

Conclusions

DaQuVa demonstrates that a compact domain-specific language can express common data-quality workflows over tabular data without requiring users to write SQL or general-purpose Python scripts. The current prototype covers connection declaration, table scanning, filtering, duplicate detection, merging, export, controlled persistence, and an aggregate `summarize` command that emits a proof table of before/after row counts for an end-to-end pipeline.

The latest implementation work improves both expressiveness and usability. Compound filters now support AND / OR logic with clear precedence, string predicates cover prefix, substring, and suffix matching, and `sort` plus `limit` make exploratory top- n queries possible inside the DSL. The `summarize` command lets a pipeline emit an inspectable proof of its effect to both CSV and SQLite, closing the input–output–evidence loop expected of a data-quality tool. The interactive REPL further supports incremental development by allowing users to execute statements one at a time and inspect intermediate tables.

The runtime has also become safer and more predictable through clearer column errors, safer SQLite replacement writes, normalized CSV string values, and better handling of incompatible comparisons. These changes move DaQuVa from a syntax prototype toward an executable data-quality tool that can be demonstrated end to end on realistic cleaning scenarios.

A Appendix

A.1 PBL Problem Map

PBL PROBLEM MAP – DSL Data Quality Validator

a) Present the problem situation in 5–7 sentences

Modern companies load data into SQL databases from a wide variety of sources such as website forms, application logs, external APIs, CSV uploads, and internal services. Due to this heterogeneity, datasets almost inevitably contain quality issues, including incorrect formats, missing values, duplicates, and violations of business rules. These problems often remain hidden until the data is used for reporting, dashboards, or machine learning models. At that stage, inconsistencies lead to unstable metrics, unexpected results, and growing mistrust in data. Today, most teams address these issues using custom SQL queries or Python scripts that are difficult to maintain, reuse, and explain. Data quality rules are frequently undocumented, scattered across code, or exist only as implicit assumptions. Therefore, companies need a clear, simple, and repeatable way to define what “good data” means, identify problematic records, suggest safe corrections, and preserve original data.

b) Formulate the problem – 1 statement

There is no easy and repeatable way to define data quality rules for SQL tables, automatically detect violations, and obtain safe and explainable correction suggestions without writing custom code.

PBL PROBLEM MAP

1. Who has a problem?
Data engineers, data analysts, data scientists, and developers responsible for maintaining reliable databases and analytics workflows.
2. Who is involved in the problem situation?
All participants in the data lifecycle: applications and services that produce data, ETL engineers who move and transform it, analysts who interpret it, machine learning specialists who train models on it, and business users who rely on analytical results.
3. Who suffers from the problem?
Technical specialists suffer first, as they spend many hours debugging pipelines and cleaning data manually. Then business users suffer because report preparation takes significantly more time, which delays decision-making and increases operational overhead.
4. Who should take part in solving the problem?
At minimum, data engineers and domain or business experts should be involved. In larger setups, data product owners and compliance and/or security specialists may also participate.
5. What happened?
Data stored in the database no longer follows expected quality rules, and important columns contain incorrect, missing, or inconsistent values.

6. What is the problem?
Data quality checks are either missing or implemented through one-off scripts, which leads to late detection of issues and risky manual fixes.
7. What led you to identify the problem (reasons, causes, evidence, arguments, findings, survey results, etc.)?
We observe frequent pipeline failures, reports that contradict each other, repeated manual data cleaning before every analysis, and increasing time spent investigating data-related issues. These observations are consistently confirmed by industry research on data engineering and analytics practices.
8. What are the causes that affect those involved?
 - Data originates from many heterogeneous sources
 - Business rules and data quality constraints are poorly formalized, scattered across code, or exist only as implicit knowledge
 - SQL-level constraints and checks are hard to document, audit, and communicate to non-technical stakeholders
 - Transformations and migrations occur without sufficient validation and visibility
 - Human input errors and inconsistent data formats
9. What are the needs of those targeted?
 - Ability to define validation rules without writing code
 - Clear and understandable explanations of detected issues
 - Reasonable and transparent suggestions for fixing invalid values
 - Safe application of fixes or creation of cleaned data copies
 - Repeatable checks with a complete audit trail
10. What are the resources (financial, human, logistical, time, etc.) necessary to solve the problem?
Financial: minimal for a prototype, higher for production usage
Human: 1–2 developers and one domain expert
Logistical: access to test databases and representative datasets
Time: moderate; the main challenge lies in designing robust rules rather than database connectivity
11. Where did the problem arise (location, area, stage of the process)?
The problem appears throughout the data lifecycle, especially during ingestion and transformation stages. It becomes most critical immediately before analytics and machine learning tasks.
12. When did the problem arise?
It appeared gradually, as soon as companies started systematically collecting and storing large volumes of data for operational, analytical, and decision-making purposes.
13. When was the problem identified?
Typically when data issues begin to regularly break reports, dashboards, or pipelines, usually detected by data analysts before reaching business users.
14. When is it planned to be resolved?
Within this project, the problem is addressed once a working MVP supports core validation rules and safe fix application. In practice, data quality improvement is an ongoing, iterative process.

15. Why is there a need to solve the problem?
Poor data quality leads to delayed decisions, reduced trust in analytics, slower workflows, higher operational costs, and potential regulatory risks.
16. How (in what way) can the problem be solved?
By introducing a declarative domain-specific language (DSL) that allows users to define data quality rules explicitly. The system automatically executes validations, presents understandable error reports, suggests sensible fixes, and applies changes only after user approval.
17. How will the elimination of the problem be determined?
Through measurable improvements such as fewer invalid values, fewer pipeline failures, reduced manual data cleaning, reproducible validation results, and complete audit logs.
18. How big is the problem?
The problem is large and widespread. Even a small percentage of invalid values can severely impact analytics when they affect identifiers, dates, monetary fields, or other critical columns.
19. How much is the quantity/quality/number of people affected?
 - Quantity: number of invalid cells and affected rows
 - Quality: reliability and trustworthiness of analytics and ML models
 - People: engineers and all decision-makers who rely on data
20. How long will it take to resolve the problem?
While the problem cannot be fully eliminated, its impact can be significantly reduced. By introducing a dedicated DSL, the most frequent and critical data quality issues can be detected and resolved within minutes or hours, depending on task complexity, which is considerably faster than manual scripting approaches. The development and initial deployment of the DSL is planned to be completed by June, according to the university project timeline. Further improvements and rule extensions can then be introduced incrementally.
21. How many parties need to be involved in resolving it?
A small team of 2–4 people is sufficient for a prototype. In production environments, coordination between data owners, engineers, and data consumers is required.
22. Why is a data quality validator needed at the database layer if validation should ideally happen before data insertion?
In real-world systems, it is impossible to fully control all data entry paths, and validation rules continuously evolve. As a result, even well-designed pipelines inevitably produce corrupted or inconsistent data, which must be detected and corrected directly within the database.

How do you see solving this problem (in terms of deliverable/product/result)?

The final deliverable is a DSL-based Data Quality Validator that:

1. Connects to SQL databases
2. Accepts clear, human-readable quality rules for tables and columns
3. Detects violations and explains them in a transparent manner
4. Suggests reasonable fixes based on observed data patterns
5. Applies changes only after explicit user approval or generates cleaned and audit tables

The system acts as a formal and transparent data quality contract rather than a black-box automatic cleaner.

Conclusions, findings, reflections

- Data quality issues are permanent and can only be controlled, not fully eliminated.
- A DSL is effective because it transforms scattered scripts into readable, reusable, and auditable rules.
- Preserving original data and requiring human approval for changes is the strongest design choice.
- The main risk lies in overly aggressive automatic corrections, mitigated through transparency and user control.
- Overall, the solution provides a practical foundation for restoring trust in data used for analytics and machine learning.

A.2 Project Plan

DaQuVa: Data Quality Validator. DSL Project Plan

1 Milestones and Timeline

Table 1: Project Milestones and Timeline

Date	Milestone	Points
February 2, 2026	Domain Selection and Problem Identification	5
February 7, 2026	Domain Analysis	10
February 13, 2026	Literature Review (min. 20 references)	10
February 20, 2026	Project Plan Presentation	5
February 27, 2026	Language Syntax Design (BNF Grammar)	5
March 7, 2026	Derivation Trees for Control Structures and Variables	5
March 14, 2026	Lexer and Parser Implementation	15
March 9-21, 2026	Evaluation I (Presentation and Report)	20
March 27, 2026	Lexical Analyzer and Symbol Table Generation	15
April 10, 2026	Semantic Analysis and Extended Paper Submission	15
April-May, 2026	Evaluation II (Demo and Report)	30
May 4-16, 2026	Final Report Write-up	-
June, 2026	Final Presentation and Defense	-

2 Task Assignments

All team members contribute to design, implementation, evaluation, and documentation. Specific roles:

- Pancenco Mihail: Grammar design, BNF formalization, semantic analysis.
- Tureac Darea: Lexer implementation, parser development, AST construction.
- Cimendur Rustem: Interpreter/execution engine, SQL integration, testing.

3 Chosen Tools

Python for lexer, parser, interpreter, and execution engine. The engine generates and runs SQL code for data validation.

A.3 Team Contract

Team Contract

Project Title: DaQuVa: Data Quality Validator. DSL

Semester: 4

Project theme: Domain Specific Language (DSL) for Data Quality Validation

Project period: February - June 2026

ECTS: 8

Supervisory team: Mentor: Vladislav Crucerescu

Project group number: 16

Cimendur Rustem, FAF-243

Frunza Damian, FAF-243

Pancenco Mihail, FAF-243

Tureac Darea, FAF-243

By signing this document, each member of the group confirms participation on equal terms in the process of writing the project. Thus, each member of the group is responsible for all contents in the project.

Chisinau, 2026

Rules and Responsibilities

- All members must attend weekly team meetings and mentor sessions.
- Each member is responsible for completing assigned tasks on time, as per the project plan.
- Pancenco Mihail: Lead grammar design, contribute to implementation and documentation.
- Tureac Darea: Lead lexer and parser implementation, contribute to testing and reports.
- Cimendur Rustem: Lead execution engine and SQL integration, contribute to documentation and grammar definition.
- All members must communicate progress and issues promptly via group channels.
- Maintain project artifacts (code, references) in shared repositories (Git, Zotero).
- Rotate roles such as meeting notetaker and presentation speaker.

Penalties for Not Adhering to the Agreed-Upon Rules

- Missing a deadline: The member must complete an additional task or compensate with extra work.
- Unexcused absence from meetings: Treat the team to pizza or snacks at the next meeting.
- Repeated breaches (e.g., 3+ instances): Escalate to mentor for review, potential team reshuffle or grade adjustment.

Bibliography

- [1] Rahm, E.; Do, H. H. Data Cleaning: Problems and Current Approaches. *IEEE Bulletin of the Technical Committee on Data Engineering*, volume 23, no. 4, 2000: pp. 3–13. Available from: <https://db.uni-leipzig.de/files/research/publications/2000-1/pdf/TBDE2000.pdf>
- [2] Chu, X.; Ilyas, I. F.; et al. Data Cleaning: Overview and Emerging Challenges. In *Proceedings of the 2016 International Conference on Management of Data (SIGMOD)*, ACM, 2016, pp. 2201–2204, doi:10.1145/2882903.2912574. Available from: <https://dbgroup.cs.tsinghua.edu.cn/jnwang/papers/sigmod2016-datacleaning-tutorial.pdf>
- [3] Raman, V.; Hellerstein, J. M. Potter’s Wheel: An Interactive Data Cleaning System. In *Proceedings of the 27th International Conference on Very Large Data Bases (VLDB)*, VLDB Endowment, 2001, pp. 381–390. Available from: <https://control.cs.berkeley.edu/pwheel-vldb.pdf>
- [4] Kandel, S.; Paepcke, A.; et al. Wrangler: Interactive Visual Specification of Data Transformation Scripts. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI)*, ACM, 2011, pp. 3363–3372, doi:10.1145/1978942.1979444. Available from: <https://idl.cs.washington.edu/files/2011-Wrangler-CHI.pdf>
- [5] International Organization for Standardization. ISO/IEC 25012:2008 — Software Engineering — Software Product Quality Requirements and Evaluation (SQuaRE) — Data Quality Model. Technical report ISO/IEC 25012:2008, ISO/IEC, 2008. Available from: <https://www.iso.org/standard/35736.html>
- [6] Heltweg, P.; Schwarz, G.-D.; et al. Can a Domain-Specific Language Improve Program Structure Comprehension of Data Pipelines? A Mixed-Methods Study. 2025, arXiv:2505.16764 [cs.PL]. Available from: <https://arxiv.org/abs/2505.16764>
- [7] Kandel, S.; Paepcke, A.; et al. Wrangler: Interactive Visual Specification of Data Transformation Scripts. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, ACM, 2011, pp. 3363–3372. Available from: <http://vis.stanford.edu/files/2011-Wrangler-CHI.pdf>
- [8] Dallachiesa, M.; Ebaid, A.; et al. NADEEF: A Commodity Data Cleaning System. In *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data*, ACM, 2013, pp. 541–552, doi:10.1145/2463676.2465327. Available from: <https://doi.org/10.1145/2463676.2465327>
- [9] Rekatsinas, T.; Chu, X.; et al. HoloClean: Holistic Data Repairs with Probabilistic Inference. *Proceedings of the VLDB Endowment*, volume 10, no. 11, 2017: pp. 1190–1201, doi:10.14778/3137628.3137631. Available from: <https://doi.org/10.14778/3137628.3137631>

- [10] Yakout, M.; Ouzzani, M.; et al. Interactive and Deterministic Data Cleaning. In *Proceedings of the 2016 International Conference on Management of Data (SIGMOD)*, ACM, 2016, pp. 1205–1217. Available from: <https://dl.acm.org/doi/10.1145/2882903.2915242>
- [11] Friedrich-Alexander Universität Erlangen-Nürnberg. Jayvee: A Domain-Specific Language for Data Pipelines. 2026, accessed: 2026-02-14. Available from: <https://jvalue.github.io/jayvee/>
- [12] Langer, P.; Kappel, G.; et al. Quality Assessment and Quality Improvement for UML Models. *Software and Systems Modeling*, volume 13, no. 2, 2014: pp. 571–594. Available from: <https://www.swe.informatik.uni-goettingen.de/publications/quality-assessment-and-quality-improvement-uml-models>
- [13] OpenRefine Community. OpenRefine. 2012, open-source tool for cleaning and transforming messy data. Available from: <https://openrefine.org/>
- [14] Sukhobok, D.; Nikolov, N.; et al. Tabular Data Cleaning and Linked Data Generation with Grafter-izer. In *Proceedings of the 13th European Semantic Web Conference (ESWC 2016), Lecture Notes in Computer Science*, volume 9989, Springer, 2016, pp. 134–139, doi:10.1007/978-3-319-47602-5_27. Available from: https://doi.org/10.1007/978-3-319-47602-5_27
- [15] Alteryx, Inc. Wrangle Language Reference. 2024, accessed: 2026-02-14. Available from: <https://help.alteryx.com/aac/en/trifacta-classic/wrangle-language.html>